

CRAMER MÖBEL

Der CRAMER PLANER, technisch erklärt

Wie das Konfigurations-Tool aufgebaut ist, wo die Daten liegen, wie ein Preis zustande kommt – und was der Kunde später selbst pflegen kann.

QUELLE	Quellcode CRAMER PLANER - Kopie für Johannes, Cramer-Stammdaten.xlsx, ARTIKELNUMMER-LOGIK.md
DATENSTAND	190 Artikel · 1.509 Preiszeilen · 9 Serien · Preisliste 06.2026
ZIELGRUPPE	Projektteam, Vertrieb, Geschäftsführung, Kunde – keine Softwareentwickler
STAND	18. August 2026

Inhalt

CRAMER PLANER — Technischer Aufbau, verständlich erklärt	4
1 Gesamtarchitektur	5
1.1 Das Bild in einem Satz	5
1.2 Das Schema	5
1.3 Die Bestandteile im Einzelnen	6
1.4 Wie die Teile miteinander sprechen	8
1.5 Was in der Oberfläche liegt, was im Hintergrund	8
2 Ablauf einer Konfiguration	10
3 Datengetriebene Architektur	14
3.1 Die wichtigste Aussage vorweg	14
3.2 Die entscheidende Unterscheidung: was ist Daten, was ist Code	14
3.3 Die Stammdatenfelder im Einzelnen	15
3.4 Wie man Daten pflegt — die drei Wege	18
4 Der tatsächliche Code — Datei für Datei, Funktion für Funktion	20
4.0 Drei Begriffe vorweg	20
4.1 src/lib/modus.ts — die Serien-Freigabe	20
4.2 src/lib/stammdaten.ts — die Anlaufstelle für alle Stammdaten	20
4.3 src/lib/stammdatenStore.ts — der veränderbare Arbeitsstand	21
4.4 src/lib/raster.ts — Zentimeter ↔ Raster	22
4.5 src/lib/preisAchsen.ts — die Preistabelle lesen	23
4.6 src/lib/preisLookup.ts — Artikelnummer + Maße → Preiszelle	23
4.7 src/config/preisMapping.ts — die Übersetzungstabelle	24
4.8 src/lib/kalkulation.ts — die Kalkulations-Engine	25
4.9 Die Regel-Dateien der Oberfläche	27
4.10 Zustand und Speicherung	28
4.11 Ausgabe und Schnittstellen	28
4.12 Reste aus der Vorgänger-Fassung	29
5 Zusammenspiel von Oberfläche, Daten und Logik	30
5.1 Beispiel: Der Benutzer klickt auf „Griff“	30

5.2 Beispiel: Der Benutzer wählt eine Serie	30
5.3 Beispiel: Der Benutzer wählt ein Material	31
5.4 Beispiel: „Ausstattung hinter der Front“	31
6 Regeln und Fehlervermeidung	33
6.1 Filterregeln — was gar nicht erst erscheint	33
6.2 Kompatibilitätsregeln — was sich gegenseitig ausschließt	33
6.3 Pflichtfelder — erzwungene Progression	34
6.4 Plausibilitätsprüfungen — Hinweise statt Sperren	34
6.5 Preislogik — nie raten	35
6.6 Gesperrte Artikel	35
6.7 Prüfungen beim Erzeugen der Stammdaten	35
7 Ein vollständiges Beispiel	36
8 Wenn der Kunde fragt	42
9 Glossar	45
10 Die kompakte 2-Minuten-Erklärung	47
Anhang: Die wichtigsten Prüf- und Pflegebefehle	47
Anhang: Zahlen zum aktuellen Datenbestand	48

CRAMER PLANER — Technischer Aufbau, verständlich erklärt

Zweck dieses Dokuments: Projektteam, Vertrieb und Geschäftsführung sollen Rückfragen des Kunden zum technischen Aufbau selbst beantworten können — ohne Programmierkenntnisse.

Stand der Analyse: Quellcode-Ordner CRAMER PLANER - Kopie für Johannes, Stammdatenmappe Cramer-Stammdaten.xlsx (190 Artikel · 1.509 Preiszeilen) und ARTIKELNUMMER-LOGIK.md.

Grundregel für dieses Dokument: Es wird ausschließlich beschrieben, was tatsächlich im Code und in den Daten steht. Wo etwas nicht eindeutig aus dem Code hervorgeht oder noch nicht umgesetzt ist, steht das ausdrücklich dabei. Drei Markierungen ziehen sich durch den Text:

Markierung	Bedeutung
[IST]	So ist es heute im Code umgesetzt und nachprüfbar.
[LÜCKE]	Technisch vorbereitet, aber noch nicht (vollständig) angeschlossen.
[EMPFEHLUNG]	Meine Einschätzung, nicht der aktuelle Stand.

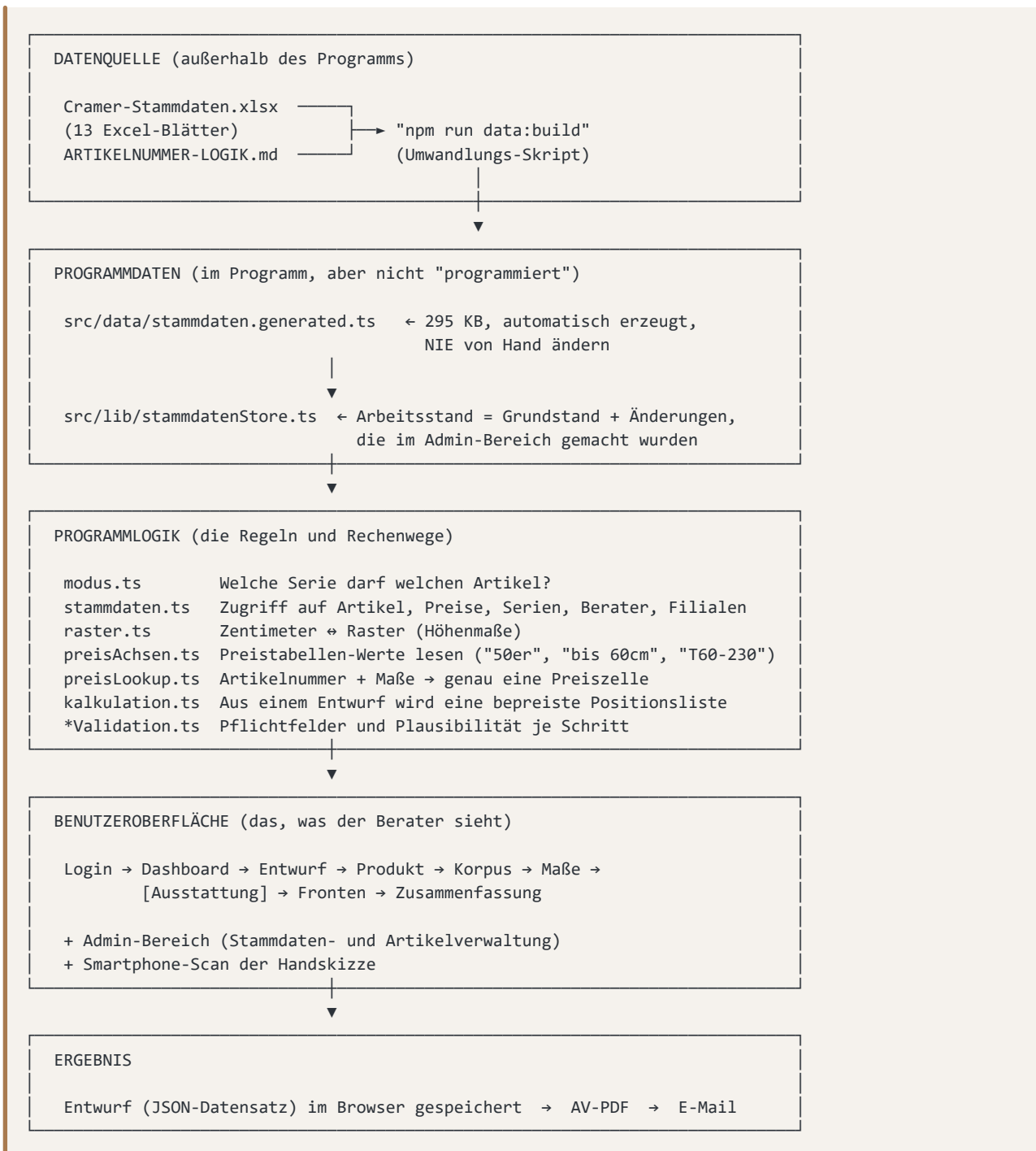
KAPITEL 1

Gesamtarchitektur

1.1 Das Bild in einem Satz

Der CRAMER PLANER ist eine **Web-Anwendung, die vollständig im Browser läuft** (Laptop oder iPad, im Kundengespräch). Es gibt **keinen Server und keine Datenbank**. Alle Produktdaten kommen aus einer ExcelMappe, die vor dem Start in eine Programmdatei umgewandelt wird. Alles, was ein Berater eingibt, wird lokal im Browser gespeichert.

1.2 Das Schema



1.3 Die Bestandteile im Einzelnen

A) Die Datenquellen — hier liegen die Produktdaten

Datei	Was steht drin
Cramer-Stammdaten.xlsx	Die eigentliche Produktwelt: 190 Artikel, 1.509 Preiszeilen, 9 Serien, 9 Produktgruppen, 38 Artikelgruppen, 7 Teilearten, 12 Preislogiken, 7 Achsen, Mitarbeiter, Filialen, Kunden, Kopfdaten. Verteilt auf 13 Blätter.
ARTIKELNUMMER-LOGIK.md	Beschreibt den Aufbau der Artikelnummer und die Klassifikationstabellen. Dient als Gegenprobe zur Excel: Das Umwandlungs-Skript vergleicht beide und meldet jede Abweichung.

Beide Dateien liegen **eine Ebene über dem Programmordner**, direkt im Projektordner.

Die 13 Excel-Blätter (Namen exakt so in `scripts/lib/paths.js` hinterlegt):

00 Anleitung	10 Artikel	20 Preise	30 Programme
31 Produktgruppen	32 Artikelgruppen	33 Teilearten	34 Preislogiken
35 Achsen	40 Mitarbeiter	41 Filialen	42 Kunden
50 Meta			

B) Das Umwandlungs-Skript — die Brücke von Excel ins Programm

`scripts/build-stammdaten.js` liest beide Quelldateien und schreibt daraus die Datei `src/data/stammdaten.generated.ts`. Das läuft **automatisch** vor jedem Programmstart (`npm run dev`) und vor jedem Erstellen einer neuen Programmversion (`npm run build`).

Das Skript macht dabei drei Dinge gleichzeitig:

1. **Daten übernehmen** — jede Excel-Zeile wird zu einem Datensatz im Programm.
2. **Typen erzeugen** — aus den Wertelisten-Blättern entstehen feste Wertevorräte. Wer später im Code `'REFUGIUM'` statt `'refugium'` schreibt, bekommt sofort einen Fehler gemeldet, statt dass die Auswahlliste stillschweigend leer bleibt.
3. **Kreuzprüfung** — Excel gegen Markdown, und alle Verweise innerhalb der Mappe gegeneinander. Beispiele für Meldungen, die das Skript ausgibt: „Artikel X hat eine unbekannte Artikelgruppe“, „Preiszeilen verweisen auf einen Artikel, den es nicht gibt“, „Mitarbeiter verweist auf eine Filiale, die es nicht gibt“, „Modus gibt keine Serie frei — der Artikel erscheint nirgends“.

C) Die Programmdaten — Grundstand und Arbeitsstand

- `src/data/stammdaten.generated.ts` ist der **Grundstand**. Er ist ein reines Erzeugnis und wird bei jedem Build überschrieben. Deshalb darf er niemals von Hand geändert werden.
- `src/lib/stammdatenStore.ts` ist der **Arbeitsstand**. Er legt sich wie eine Folie über den Grundstand: Was ein Administrator in der App ändert, wird als *Abweichung* gespeichert (geändert / neu / gelöscht), nicht als Kopie des gesamten Datenbestands.

Vorteil: Wird später eine neue Excel importiert, kommen alle unveränderten Zeilen frisch mit, und die eigenen Änderungen bleiben als klar benennbare Liste sichtbar. Die Frage „Wer hat was geändert?“ ist damit jederzeit beantwortbar. Die App zeigt oben rechts „x geändert“.

D) Die Programmlogik — die Regeln

Alle Rechen- und Regelwege liegen in `src/lib/`. Diese Dateien haben **keine Oberfläche**; sie arbeiten im Hintergrund. Die wichtigsten:

Datei	Aufgabe
<code>modus.ts</code>	Liest die Serien-Freigabe eines Artikels (Feld „Modus“).
<code>stammdaten.ts</code>	Einzige Anlaufstelle für Artikel, Preise, Serien, Berater, Filialen.
<code>raster.ts</code>	Rechnet Zentimeter in Rasterstufen um (Höhenmaße).
<code>preisAchsen.ts</code>	Versteht die uneinheitlichen Werte der Preistabelle.
<code>preisLookup.ts</code>	Findet zu Artikelnummer + Maßen genau eine Preiszelle.

Datei	Aufgabe
kalkulation.ts	Baut aus einem Entwurf die bepreiste Positionsliste.
korpusMass.ts	Rechnet die Korpus-Grunddaten in Maße und Koordinaten um.
*Validation.ts / *Rules.ts	Pflichtfelder und Plausibilitätsregeln je Schritt.
generatePdf.ts	Erzeugt das AV-PDF.

E) Die Benutzeroberfläche

Alles unter `src/pages/` (ein Ordner je Schritt) und `src/components/` (wiederverwendbare Bausteine wie die MaterialAuswahl oder eine Front-Karte). Diese Dateien zeigen an und nehmen entgegen — sie **rechnen nicht selbst**.

F) Die Konfigurationsdateien — eine Zwischenwelt

In `src/config/` liegen Kataloge, die zwar **im Code** stehen, aber wie Daten aufgebaut sind (reine Listen, keine Logik). Das ist der wichtigste Punkt zum Verständnis des heutigen Stands:

Datei	Inhalt	Kommt aus
productCatalog.ts	Produktfamilien-Kacheln (Sideboards, Kleiderschränke ...) und je Familie die zugelassenen Serien	Code (Serien-Namen aus der Mappe)
materialMatrix.ts	Die vollständige Farb-/Material-/Preisgruppenmatrix	Code (Dokument „CRAMER PLANER - Farbmatrix“)
frontCatalog.ts	Front-Typen, Stil-Linien und deren Materialfelder	Code (Fronten-Doku)
equipment.ts	Ausstattungs-Katalog für Refugium	Code (Fachberater-PDF)
handles.ts	Griff-Auswahlliste	Code
korpus.ts	Korpus-Bereiche (Innen/Außen/Abdeckplatte) und ihre Materialzulassungen	Code
preisMapping.ts	Übersetzungstabelle: Konfigurator-Begriff → Artikelnummer	Code
workflow.ts	Die Schrittfolge des Konfigurators	Code

1.4 Wie die Teile miteinander sprechen

- Die Oberfläche schreibt **alles** in ein einziges Objekt: den **Entwurf** (Draft). Dieser Entwurf ist die zentrale Wahrheit. Jeder Schritt ergänzt ihn.
- Die Kalkulation liest den Entwurf, übersetzt die Auswahlen über `preisMapping.ts` in Artikelnummern und fragt damit `preisLookup.ts` nach Preisen.
- Das PDF liest denselben Entwurf.
- Die Oberfläche liest die Stammdaten immer über `stammdaten.ts` bzw. `stammdatenStore.ts` — nie direkt aus der erzeugten Datei. Deshalb wirkt eine Änderung in der Artikelverwaltung **sofort** in Auswahllisten und Kalkulation, ohne Neuladen.

1.5 Was in der Oberfläche liegt, was im Hintergrund

Im Vordergrund (sichtbar)	Im Hintergrund (unsichtbar)
Seiten unter <code>src/pages/</code>	Regeln unter <code>src/lib/</code>
Bausteine unter <code>src/components/</code>	Stammdaten-Store und Preis-Lookup
Auswahllisten, Kacheln, Eingabefelder	Kalkulations-Engine
Zusammenfassung, PDF-Vorschau	PDF-Erzeugung, Excel-Import/-Export
Admin-Tabellen	Kreuzprüfung beim Build

KAPITEL 2

Ablauf einer Konfiguration

Der Weg durch das Programm ist als Datenliste hinterlegt (`src/config/workflow.ts`) und nicht fest verdrahtet — ein weiterer Schritt lässt sich ergänzen, ohne die Anzeige umzubauen.

```
Entwurf → Produkt → Korpus → Maße → [Ausstattung] → Fronten → Abschluss  
/new      /products /korpus /dimensions /ausstattung /fronts /summary
```

Der Schritt „Ausstattung“ erscheint **nur bei der Serie Refugium** (Kleiderschränke). Geregelt in `visibleWorkflowSteps()`.

Im Folgenden: je Schritt, was der Benutzer sieht — was technisch passiert — welche Daten benutzt werden.

Schritt 0 — Anmeldung (/login)

Der Benutzer sieht: E-Mail und Passwort.

Technisch: `AuthContext.tsx` prüft die Eingabe. Die Beraterliste kommt aus dem Excel-Blatt „40 Mitarbeiter“ — aber nur Zeilen mit Rolle `berater` und Status `aktiv` (`src/lib/stammdaten.ts`, Konstante `berater`).

Daten: Blatt „40 Mitarbeiter“. **Passwörter stehen bewusst NICHT in der Mappe.** Für den Prototyp gilt ein gemeinsames Demo-Passwort (`src/data/consultants.ts`); zusätzlich gibt es einen getrennten Zugangs-Speicher mit SHA-256-Hashes (`src/lib/zugangStore.ts`), damit Administratoren Berater-Zugänge vergeben können, ohne dass ein Klartext-Passwort irgendwo landet.

Schritt 1 — Dashboard (/)

Der Benutzer sieht: Liste aller Entwürfe, Suche nach Auftrags-/Artikelnummer/Kunde, Filter nach Berater und Filiale, Buttons „Fortsetzen“, „Duplizieren“, „Löschen“.

Technisch: `Dashboard.tsx` liest die gespeicherten Entwürfe aus dem `DraftContext`. Die Funktion `resumeRoute()` ermittelt, wo ein Entwurf weitergehen muss: Sie ruft der Reihe nach `isKorpusComplete`, `isDimensionsValid`, `isFrontsComplete` auf und springt zum ersten unvollständigen Schritt.

Daten: Der Browser-Speicher (`localStorage`, Schlüssel `cramer-planer.drafts.v2`) und die Filialliste aus Blatt „41 Filialen“.

Schritt 2 — Entwurf anlegen (/new)

Der Benutzer sieht: Kundenname (Pflicht), Filiale (Pflicht), Auftragsnummer und Artikelnummer (beide optional).

Technisch: `startNewDraft()` erzeugt den Entwurf mit einer automatischen Entwurfsnummer. `generateEntwurfsnummer()` in `src/lib/id.ts` baut sie nach dem Muster `CRAMER-<Jahr>-<Initialen des Beraters>-<4-stellige laufende Nummer>`, z. B. `CRAMER-2026-AB-0001`. `validateDraftForm()` prüft in Echtzeit die beiden Pflichtfelder.

Daten: Filialen aus Blatt „41 Filialen“; die laufende Nummer aus einem Zähler im Browser-Speicher.

[LÜCKE] Der Nummernzähler ist **geräteweit**, nicht firmenweit. Zwei Arbeitsplätze können theoretisch dieselbe Nummer vergeben. Im Code ausdrücklich als Prototyp-Grenze vermerkt.

Schritt 3 — Produktgruppe und Serie (/products)

Der Benutzer sieht: Kacheln der Produktfamilien (Sideboards, Kleiderschränke, Regale/Wohnwände, Medienmöbel, Tische; „Sitzen“ und „Schlafen“ sichtbar aber grau). Nach der Wahl einer Familie erscheinen darunter **nur die zugelassenen Serien**. Unter der Serie steht ein Hinweis wie:

„Serien-Kürzel **A · 83** von 190 Artikeln freigegeben, verteilt auf **7** Schritte: Korpus & Struktur (2) · Fronten (5) · Frontausstattung (2) · Innenausstattung (5) ...“

Technisch:

- Die Familien-zu-Serien-Zuordnung steht in `productCatalog.ts`.
- Ein Wechsel der Familie setzt die Serie zurück (`selectGroup`) — es kann nie eine familienfremde Serie stehenbleiben.
- Der Zahlenhinweis entsteht **live aus den Excel-Daten**: `artikelFuerSerie(seriesId)` filtert alle Artikel, deren Feld „Modus“ den Serien-Buchstaben enthält und deren Status aktiv ist.
- Weiter geht es nur, wenn `getSeries(gruppe, serie)` einen Treffer liefert.

Daten: Serien aus Blatt „30 Programme“, Artikel aus Blatt „10 Artikel“ (Feld Modus + Status).

Schritt 4 — Korpus (/korpus)

Der Benutzer sieht: Umschalter „Außenkorpus komplett / getrennt“, dann je Bereich (a. Innen, b. Außen, c. Abdeckplatte) eine Reihe von Material-Chips (Decoboard, Mattlack, Furnier, Gläser, Xtreme Plus, „anders“) und darunter ein Ausführungs-Dropdown mit den konkreten Farben. Rechts steht dezent „Preisgruppe 2“.

Technisch:

- `getVisibleKorpusAreas(serie, modus)` entscheidet, welche Bereiche überhaupt erscheinen: „Innen“ nur bei Velare/Refugium; „Abdeckplatte“ verschwindet bei Serien mit `hasAbdeckplatte: false` (Refugium); „getrennt“ ersetzt „Außen“ durch „Außen links“ und „Außen rechts“.
- `MaterialSelect.tsx` baut aus den erlaubten Gruppen-IDs die Chips und aus der gewählten Gruppe das Dropdown — **immer aus derselben zentralen Farbmatrix**.
- Nach der Wahl setzt `resolvePriceGroup()` die Preisgruppe automatisch: Options-Preisgruppe hat Vorrang vor Gruppen-Preisgruppe (Beispiel: „Wenge dunkel“ ist PG4, obwohl die Gruppe Furnier PG3 ist).
- Bei „anders“ muss die Preisgruppe **manuell** gewählt werden; solange keine gewählt ist, steht „Preisgruppe offen“.
- Weiter geht es nur, wenn `isKorpusComplete()` alle sichtbaren Pflichtbereiche als vollständig meldet.

Daten: `materialMatrix.ts` (Code), `korpus.ts` (Code). **Nicht** aus der Excel.

Schritt 5 — Maße (/dimensions)

Hier gibt es **zwei verschiedene Oberflächen**, gesteuert über das Serien-Flag `korpusRaster`:

Refugium (strukturiert, `KorpusMasseSection`): Höhe als Auswahl 18 Raster (ca. 235 cm) / 21 Raster (ca. 274 cm) / anders; Tiefe 60 cm / anders; danach **je Korpus** eine Zeile mit Breite (50er / 60er / 100er / anders), Häkchen „Lochreihe“ und „Fixmaß“; darunter Abschlusset (keine / links / rechts / beide + Material), Fußleistenausschnitt und ein Freitextfeld „Sonderformen“.

Alle anderen Serien (einfach): Höhe, Breite, Tiefe als Zahlenfelder plus ein Zähler „Anzahl Segmente“ (1–12).

Technisch:

- Aus den strukturierten Grunddaten leitet `deriveDimensions()` die einfachen Maße ab, damit Fronten, Zusammenfassung und PDF unverändert weiterlaufen.
- `computeKorpusKoordinaten()` berechnet je Korpus die linke Kante als laufende Summe der Breiten — die Vorbereitung für eine spätere 2D-/3D-Darstellung.
- Beim Klick auf „Weiter“ erzeugt `syncColumns()` **je Segment eine Front-Spalte**. Bereits gefüllte Spalten bleiben erhalten, fehlende kommen dazu, überzählige rechts fallen weg.
- Refugium springt danach zu `/ausstattung`, alle anderen Serien direkt zu `/fronts`.

[LÜCKE] `computeKorpusMasse()` in `korpusMass.ts` ist ausdrücklich ein **Platzhalter**. Die exakte Ableitung des Außenmaßes (Frontbreite → Korpusbreite, +3 mm Luft je Fuge, +10 mm je Abdeckplatte, AbschlussetZuschläge) ist noch nicht hinterlegt; die Konstanten stehen bereits vorbereitet in `MASS_KONSTANTEN`. Das Feld `berechnet` steht deshalb auf `false`, und die Maße werden als „ca.“-Werte ausgewiesen.

Schritt 6 — Ausstattung-Vorauswahl (`/ausstattung`, nur Refugium)

Der Benutzer sieht: Häkchen-Listen in fünf Kategorien (Essentials, Verblendung, Beleuchtung, Ausstattung-Craft, Ausstattung-Conero). „Einlegeboden“ und „Einlegeboden inkl. Kleiderstange“ sind vorausgewählt.

Technisch: Der Katalog steht in `equipment.ts`. Erkennt das Programm eine **Sondertiefe** (< 60 cm, ermittelt durch `isSondertiefeDepth()`), werden automatisch alle Optionen außer dem einfachen Einlegeboden deaktiviert **und aus der Auswahl entfernt** — die Regel „Bei Sondertiefen nur Einlegeböden“ ist damit nicht nur ein Hinweis, sondern erzwungen.

Diese Vorauswahl wirkt als **Filter für Schritt 8**: Nur was hier angehakt ist, wird später hinter den Fronten überhaupt angeboten.

Schritt 7 — Fronten (`/fronts`)

Der Benutzer sieht: Je Segment einen Block „Front-Typ 1 ... n“ mit Positionsangabe („ganz links“, „ganz rechts“). Darunter eine Knopfleiste „+ Drehtür / + Schiebetür (einläufig) / + Schübe / + Stauraumklappe / + Hochstellklappe / + Schreibklappe / + Offen (Regal)“. Bei Refugium zusätzlich „+ Schiebetür (zweiläufig)“.

Jedes hinzugefügte Bauteil wird zu einer Karte mit: Kennzeichnung (Pflicht, z. B. „D1“), Breite/Höhe/Katalog-Code, Stil-Linien-Chips (Glatt, Glossy, Less, Line, 107, Curve), den zur Stil-Linie gehörenden Material-Auswahlen, und — bei Glatt/Glossy/Less — den Griff-Optionen (PTO oder Griff, gegenseitig ausschließend) samt Griff-Dropdown.

Technisch:

- `getAvailableFrontTypes(serieId)` blendet die zweiläufige Schiebetür für alle Serien außer Refugium aus.
- `canAddFrontType()` setzt die Exklusivitäts-Regel um: Ist eine zweiläufige Schiebetür geplant, lässt sich **gar nichts mehr** hinzufügen; umgekehrt lässt sie sich nur einfügen, wenn noch keine andere Front existiert.

- `schiebetuerAnzahlOptions(segmentzahl)` bestimmt die zulässige Türanzahl: 2 Korpi → 2 · 3 Korpi → 3 · 4 Korpi → 2 oder 4.
- Ein Wechsel der Stil-Linie leert die Materialfelder (sie sind je Linie anders aufgebaut) und verwirft einen Griff, der in der neuen Linie nicht mehr zulässig ist.
- `hideWhenSiblingGroupIn` blendet bei 107/Curve das Freitextfeld „Griffleiste gepulvert in:“ aus, sobald als Material Decoboard oder Xtreme Plus gewählt wurde.
- Bei Refugium erscheint pro Segment ein aufklappbarer Bereich „Ausstattung hinter der Front“ — aber nur, wenn das Segment eine Drehtür, eine zweiläufige Schiebetür oder ein offenes Fach enthält (`EQUIPMENT_ELIGIBLE_FRONT_TYPES`).

Daten: `frontCatalog.ts`, `materialMatrix.ts`, `handles.ts`, `equipment.ts` — alle im Code.

Schritt 8 — Zusammenfassung und Abschluss (/summary)

Der Benutzer sieht: Alle Angaben untereinander, den Scan-Bereich für die Handskizze, das **Kalkulations-Panel** mit der vollständigen Positionsliste, das Feld „Verkaufspreis (VK)“ und vier Knöpfe: Zurück · PDF herunterladen · An AV senden · Speichern & abschließen.

Technisch:

- `KalkulationsPanel` ruft `berechneEntwurf(draft)` auf und zeigt jede Position mit Artikelnummer, Kurzzeichen, Klassifikation, Achsenwerten und Seite der Preisliste.
- „Berechneten Preis übernehmen“ schreibt die Summe in das VK-Feld.
- **Der VK-Preis ist der verbindliche Preis** — die Kalkulation ist ein Vorschlag.
- „An AV senden“ ist gesperrt, solange Auftragsnummer **oder** Artikelnummer fehlt. Diese beiden Felder sind bewusst erst hier Pflicht, nicht bei der Anlage.
- `downloadPdf(draft)` erzeugt das AV-PDF (Seite 1 Daten, Seite 2 die gescannte Skizze) und öffnet anschließend eine vorbereitete E-Mail.

KAPITEL 3

Datengetriebene Architektur

3.1 Die wichtigste Aussage vorweg

Wir konfigurieren nicht fertige Möbel, sondern setzen ein Möbel aus einzelnen Artikeln zusammen — wie bei einem Fahrzeugkonfigurator.

Das ist im Code konsequent umgesetzt: Aus einem Refugium-Schrank mit vier Segmenten entstehen 17 einzelne Positionen (4× Korpus, 3× Mittelseite, 1× Außenset, 5 Fronten, 1 Griff, Innenausstattung ...), jede mit eigener Artikelnummer und eigenem Preis. Es gibt keinen Artikel „Kleiderschrank“.

3.2 Die entscheidende Unterscheidung: was ist Daten, was ist Code

Der CRAMER PLANER hat heute **zwei Datenwelten**. Das muss man wissen, um Kundenfragen richtig zu beantworten.

Welt 1 — Der Artikelstamm (Excel, pflegbar ohne Programmierung)

Hier liegt alles, was den **Artikel und seinen Preis** beschreibt:

Was	Wo	Pflegbar ohne Programmierer?
Artikelnummer, Bezeichnung, Kurzzeichen	Blatt „10 Artikel“	Ja
Teileart, Produktgruppe, Artikelgruppe	Blatt „10 Artikel“	Ja
Modus (Serien-Freigabe)	Blatt „10 Artikel“	Ja
Status aktiv / gesperrt / entwurf	Blatt „10 Artikel“	Ja
Preislogik, Einheit, Achsen, Sortierung	Blatt „10 Artikel“	Ja
Preise (1.509 Zeilen)	Blatt „20 Preise“	Ja
Serien	Blatt „30 Programme“	Ja
Berater, Filialen	Blätter „40“/„41“	Ja
Rastermaß, MwSt., Zuschlagssätze, Höhen-Offsets	Blatt „50 Meta“	Ja

Welt 2 — Die Konfigurationskataloge (im Code, aber datenförmig)

Hier liegt alles, was die **Auswahlführung im Gespräch** beschreibt:

Was	Wo	Pflegbar ohne Programmierer?
Farben und Materialien	config/materialMatrix.ts	Nein
Front-Typen und Stil-Linien	config/frontCatalog.ts	Nein
Ausstattungs-Katalog (Refugium)	config/equipment.ts	Nein
Griff-Auswahlliste	config/handles.ts	Nein

Was	Wo	Pflegbar ohne Programmierer?
Korpus-Bereiche und Materialzulassungen	config/korpus.ts	Nein
Produktfamilien-Kacheln	config/productCatalog.ts	Nein
Übersetzung Konfigurator-Begriff → Artikelnummer	config/preisMapping.ts	Nein

Diese Dateien sind bewusst als reine Listen geschrieben — ohne Rechenlogik. Eine Änderung dort ist für einen Entwickler eine Sache von Minuten, aber sie **ist** eine Code-Änderung und braucht ein neues Deployment.

[LÜCKE] — die zentrale Erkenntnis dieser Analyse: Die Regel „Artikelgruppe = ein Dropdown, gefiltert über Modus und Status“ ist im Code **vollständig implementiert und getestet** (`findeArtikel`, `dropdownEintraege`, `dropdownsFuerSchritt` in `src/lib/stammdaten.ts`, Selbsttest `npm run data:test`). Sie wird heute aber nur an **einer** Stelle sichtbar genutzt: für die Zahlenanzeige auf der Produktauswahl-Seite („83 von 190 Artikeln freigegeben“).

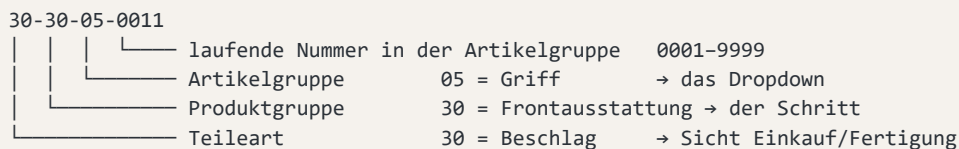
Die eigentlichen Konfigurator-Auswahllisten (Material, Front-Typ, Stil-Linie, Griff, Ausstattung) werden **noch** aus den Code-Katalogen der Welt 2 gespeist. Die Verbindung zum Artikelstamm entsteht erst bei der Bepreisung, über `preisMapping.ts`.

Das ist kein Fehler, sondern der aktuelle Ausbaustand: Die Mechanik steht, der Umbau der Auswahllisten auf den Artikelstamm ist der nächste logische Schritt.

3.3 Die Stammdatenfelder im Einzelnen

Artikelnummer — der Schlüssel

Aufbau (aus `ARTIKELNUMMER-LOGIK.md`, im Code als `artikelnummerLogik` hinterlegt):



Gelesen: *Beschlag, im Schritt Frontausstattung, Dropdown Griff, elfter Artikel.*

- Die drei Klassifikationsblöcke sind in **Zehner- bzw. Fünferschritten** vergeben. Eine neue Klasse lässt sich dazwischenschieben, ohne eine bestehende Nummer anzufassen.
- Die laufende Nummer ist fortlaufend **ohne** Lücken — die Reihenfolge im Dropdown regelt die eigene Spalte Sortierung. Damit ist **Identität von Anzeige getrennt**: Ein Artikel kann in der Liste nach oben rutschen, ohne die Nummer zu wechseln.
- Die Artikelnummer wird nie geändert.** Im Code erzwungen: `aendereArtikel()` ignoriert eine mitgegebene Nummer; im Detail-Fenster ist das Feld nur beim Anlegen beschreibbar.
- Präfix-Suche funktioniert wie im ERP: 30- alle Beschläge, 20-20-05- alle Drehtüren (`sucheNachPraefix`).

Artikelbezeichnung

Zwei Felder: `bezeichnung` (der Anzeigename) und `bezeichnung2` (die Gruppenüberschrift aus der Preisliste, z. B. „Ausstattungsreihe Conero“). Zusätzlich das **Kurzzeichen** (`DRT-001`) als reine Lesehilfe — im Code ausdrücklich als

„KEIN Schlüssel“ gekennzeichnet.

Teileart

Sieben Werte, die Sicht von Einkauf und Fertigung:

Nr	Code	Bedeutung
10	STRUKTUR	Trägt das Möbel: Korpus, Seiten, Böden, Rückwand, Sockel, Abdeckplatte
20	FRONT	Verschließt das Möbel: Türen, Klappen, Schubfronten
30	BESCHLAG	Griffe, Schlösser, Füße, Rollen
40	AUSSTATTUNG	Innenausbau: Böden, Stangen, Container, Spiegel
50	TECHNIK	Beleuchtung, Schalter, Elektrifizierung
60	KOMPLETT	Fertiges Möbel zum Festpreis (Porticus, Supersonus)
70	KALKULATION	Zuschläge, Montage, Lieferung, Sonderleistungen

Produkt-/Möbelfamilie bzw. Modus — Achtung, zwei verschiedene Dinge

Hier gibt es eine **Begriffsfalle**, die im Code ausdrücklich kommentiert ist:

- „**Produktgruppe**“ in der **Excel** meint einen *Schritt im Konfigurator* (KORPUS, FRONT, FRONTAUSSTATTUNG, INNENAUSSTATTUNG, TECHNIK, ABSCHLUSS, MOEBEL, TISCH, KALKULATION).
- „**Produktgruppe**“ auf der **Kachel-Seite** meint die *Möbelfamilie* (Sideboards, Kleiderschränke, Regale, Medienmöbel, Tische). Diese Zuordnung steht in `productCatalog.ts` im Code.
- **Die Möbelserie** (Atrium, Velare, ...) steht im Feld `Modus` des Artikels.

Der Modus — die Serien-Freigabe

Jeder Artikel trägt die Kürzel der Serien, für die er freigegeben ist:

Kürzel	Serie	Schwerpunkt
A	Atrium	Schränke, Einbauschränke, Sideboards
V	Velare	Low-, Side-, Highboards, Kommoden
P	Publicum	Bibliotheken, Regale, offene Gestaltung
R	Refugium	Kleiderschränke, begehbare Schränke
O	Porticus	Low-/Side-/Highboard-Serie auf Gestell
C	Cavum	Medienmöbel
S	Supersonus	Medienmöbel mit Akustik
T	Tavolo	Holztischsystem
U	Arcum	Brückentische / Korpuseinsätze

GROSSBUCHSTABE = Standard · kleinbuchstabe = Sonderanfertigung (Preis auf Anfrage).

Die Auswertung (`src/lib/modus.ts`) ist bewusst **maximal fehlertolerant**, weil die Werte von Hand in Excel gepflegt werden:

RP ≡ R,P ≡ R/P ≡ R - P ≡ _R_P_ ≡ p r → alle identisch

Reihenfolge, Trennzeichen und Groß-/Kleinschreibung spielen für die Frage „ist der Artikel freigegeben?“ keine Rolle. Für die Frage „Standard oder Sonderanfertigung?“ spielt die Schreibweise sehr wohl eine Rolle — dafür gibt es eine eigene Funktion.

Aktuelle Verteilung (aus den echten Daten, Status aktiv):

Serie	freigegebene Artikel	betroffene Artikelgruppen
Refugium (R)	109	25
Atrium (A)	83	22
Publicum (P)	83	22
Velare (V)	80	20
Porticus (O)	66	16
Supersonus (S)	49	12
Tavolo (T)	43	8
Cavum (C)	29	6
Arcum (U)	24	6

Preisgruppe

Vier Stufen, definiert im Blatt „50 Meta“:

PG	Bedeutung
PG1	Cramer Decoboard Holz & Farbe
PG2	Cramer Mattlack (Standardlacke der Cramer-Kollektion)
PG3	Andere RAL-Lacke, Kirsche, Nussbaum, Eiche, Esche, Ahorn, Buche, Glas 4 mm
PG4	Sikkens/NCS/RAL-Design-Lacke, Wenge dunkel, Multiplex, Corian 6 mm, Xtreme Plus

Wichtig: Die Preisgruppe wird bei der Materialauswahl **automatisch** gesetzt und ist der eigentliche Preistreiber. Die konkrete Farbe („RAL 7016“) ist für den Preis egal — sie ist reine Fertigungsinformation für die AV. Genau deshalb funktioniert das System auch mit frei wählbaren Farben: Farben sind nicht abzählbar, Preisgruppen schon.

Oberfläche

Ein Ja/Nein-Feld (oberflaeche) je Artikel: Ist Material/Oberfläche für diesen Artikel überhaupt relevant? Beispiel: Abdeckplatte ja, Montagekosten Akustikpaneel nein.

[LÜCKE] Das Feld wird bisher **nur angezeigt** (Admin-Tabelle, Spalte „Oberfläche“ J/N). Es steuert noch keine Regel im Konfigurator.

Preis

Preise stehen **nicht am Artikel**, sondern in einem eigenen Blatt („20 Preise“). Ein Artikel hat so viele Preiszeilen, wie er Varianten hat — der Refugium-Korpus zum Beispiel sechs:

Artikel	A1 Breite	A2 Raster	Preis	Seite
10-10-05-0003	50er	18	237 €	26
10-10-05-0003	60er	18	258 €	26
10-10-05-0003	100er	18	309 €	26
10-10-05-0003	50er	21	285 €	26
10-10-05-0003	60er	21	309 €	26
10-10-05-0003	100er	21	372 €	26

Jede Preiszeile trägt außerdem die **Seite der gedruckten Preisliste** mit — der Herkunftsnachweis, der in der Positionsliste und im PDF angezeigt wird.

Alle Preise sind VK-EUR **inklusive 19 % MwSt.** (Blatt „50 Meta“). Aktueller Bestand: 1.503 feste Preise, 4 „auf Anfrage“, 2 mit Hinweis.

Status aktiv / inaktiv

Drei Werte: aktiv, gesperrt, entwurf. Nur aktiv erscheint im Konfigurator (`findeArtikel()` filtert Standard-mäßig alles andere weg). Zusätzlich prüft der Preis-Lookup den Status noch einmal und gibt bei gesperrt bzw. entwurf die Meldung „Artikel 30-30-05-0011 ist gesperrt“ statt eines Preises zurück.

[IST] Derzeit stehen alle 190 Artikel auf aktiv. Der Sperr-Mechanismus ist eingebaut, wird aber noch von keinem Artikel benutzt.

Kompatibilitätsregeln

Es gibt **keine allgemeine Regeltabelle** in der Excel, in der stünde „Artikel X passt zu Artikel Y“. Kompatibilität entsteht heute auf vier Wegen — Details in Kapitel 6.

3.4 Wie man Daten pflegt — die drei Wege

Weg 1 — In Excel, dann neu erzeugen (der Hauptweg)

```
Cramer-Stammdaten.xlsx bearbeiten
↓
npm run data:build      (oder einfach npm run dev – läuft automatisch vorher)
↓
Kreuzprüfung meldet Auffälligkeiten
↓
src/data/stammdaten.generated.ts ist neu – Änderung ist im Programm
```

Weg 2 — Im Admin-Bereich der laufenden App

Der Administrator öffnet „Stammdaten & Artikelverwaltung“ (fünf Reiter: Artikelstamm, Preisblatt, Berater, Filialen, Handbuch), sucht den Artikel, ändert ihn im Detail-Fenster und speichert. Die Änderung wirkt **sofort** — auch in einer bereits geöffneten Kalkulation.

Diese Änderungen liegen als „Abweichungs-Folie“ im Browser-Speicher, nicht in der Excel.

Weg 3 — Excel-Runde (Export → bearbeiten → Import)

Der Admin-Bereich hat „Excel exportieren“ und „Excel importieren“. Der Export erzeugt eine Mappe mit **exakt denselben Blatt- und Spaltennamen** wie die Original-Stammdaten. Man kann sie in Excel bearbeiten und unverändert zurückspielen. Der Import ordnet **über die Spaltenüberschriften** zu, nicht über die Position — wer eine Spalte verschiebt oder eine Hilfsspalte einfügt, bricht nichts.

Was passiert bei welcher Änderung

Ich will ...	Was ist zu tun	Programmierer nötig?
Einen neuen Griff ergänzen	Zeile in „10 Artikel“ (Artikelgruppe GRIFF, Modus, Status aktiv) + Zeile in „20 Preise“	Nein
Einen Artikel sperren	Status auf gesperrt setzen	Nein
Einen Preis ändern	Betrag in „20 Preise“ ändern	Nein
Einen Artikel einer weiteren Serie freigeben	Buchstaben im Feld „Modus“ ergänzen	Nein
Eine neue Serie anlegen	Zeile in „30 Programme“	Nein (siehe Einschränkung unten)
Einen neuen Berater / eine Filiale	Zeile in „40 Mitarbeiter“ / „41 Filialen“	Nein
Montage-Zuschlag von 10 % auf 12 %	Blatt „50 Meta“, Schlüssel montageZuschlagPct	Nein
Eine neue Farbe im Material-Dropdown	config/materialMatrix.ts	Ja
Einen neuen Front-Typ	config/frontCatalog.ts + config/preisMapping.ts	Ja
Eine Serie in einer Produktfamilie sichtbar machen	config/productCatalog.ts	Ja
Eine Serie automatisch bepreisen (Korpus/Mittelseite/Außenset)	config/preisMapping.ts	Ja

Einschränkung bei neuen Serien: Die Serie selbst kommt aus der Excel. Damit sie aber auf einer Produktfamilien-Kachel erscheint, braucht es einen Eintrag in `productCatalog.ts`; damit sie automatisch bepreist wird, einen Eintrag in `preisMapping.ts`. Beides ist Code.

KAPITEL 4

Der tatsächliche Code — Datei für Datei, Funktion für Funktion

4.0 Drei Begriffe vorweg

- Eine „**Function**“ ist ein kleiner Arbeitsauftrag im Programm. Sie bekommt bestimmte Informationen, verarbeitet sie und liefert ein Ergebnis zurück. Beispiel: „Hier hast du die Höhe 230 cm — sag mir die Rasterstufe.“ Antwort: „18.“
- Eine „**Datei**“ ist ein Blatt Papier, auf dem verwandte Arbeitsaufträge zusammen stehen.
- Ein „**Modul**“ ist eine Datei, die anderen Dateien ihre Arbeitsaufträge anbietet.

4.1 `src/lib/modus.ts` — die Serien-Freigabe

Rolle: Die **einzige** Stelle im ganzen Programm, an der Modus-Werte gelesen werden. Auch die UmwandlungsSkripte benutzen genau dieses Modul — dadurch kann Excel-Prüfung und Laufzeit nie auseinanderlaufen.

Funktion	Was sie macht	Bekommt	Gibt zurück
<code>modusErlaubt(modus, serienCode)</code>	Steht der Artikel für diese Serie zur Verfügung? Ignoriert alles außer Buchstaben und die Groß-/Kleinschreibung.	Modus-Text, Serien-Kürzel	Ja/Nein
<code>istSonderanfertigung(modus, serienCode)</code>	Ist die Freigabe ein Klein buchstabe, also Sonderanfertigung?	dieselben	Ja/Nein
<code>parseModus(modus, codes)</code>	Zerlegt den Modus in die enthaltenen Serien-Kürzel, doppelte weg, in kanonischer Reihenfolge.	Modus-Text, Liste gültiger Kürzel	Liste von Kürzeln
<code>normalizeModus(roh, codes)</code>	Bereinigt einen Wert zur sauberen Schreibweise (A_P_O_S_ → APOS) und meldet unbekannte Zeichen sowie Groß-/Klein-Widersprüche.	Rohwert	bereinigter Wert + Auffälligkeiten

Aufgerufen wann: Bei jedem Aufbau einer Artikelliste, beim Erzeugen der Stammdaten und beim BereinigungsSkript.

4.2 `src/lib/stammdaten.ts` — die Anlaufstelle für alle Stammdaten

Rolle: Kein anderer Programmteil greift direkt auf die erzeugte Datendatei zu.

Funktion	Was sie macht	Gibt zurück
<code>getSerie(id) / getSerieByCode(code)</code>	Serie nachschlagen	Serie oder „nicht gefunden“

Funktion	Was sie macht	Gibt zurück
serienVon(artikel)	Welche Serien gibt dieser Artikel frei?	Liste von Serien
istVerfuegbar(artikel, serieId)	Ist der Artikel für diese Serie freigegeben?	Ja/Nein
istSonderanfertigungFuer(artikel, serieId)	Nur als Sonderanfertigung freigegeben?	Ja/Nein
findeArtikel(filter)	Die Kernfunktion. Filtert nach Serie, Schritt (Produktgruppe), Dropdown (Artikelgruppe) und Status; sortiert nach Spalte Sortierung, bei Gleichstand alphabetisch (deutsche Sortierung).	Artikelliste
artikelFuerSerie(serieId)	Kurzform: alles, was diese Serie freischaltet	Artikelliste
dropdownEintraege(artikelgruppe, serieId)	Die Einträge eines Dropdowns	Artikelliste
dropdownsFuerSchritt(produktgruppe, serieId)	Welche Dropdowns hat dieser Schritt? Leere werden weggelassen — deshalb verschwindet „Abdeckplatte“ bei Refugium automatisch, weil kein Abdeckplatten-Artikel ein R im Modus trägt.	Liste von Artikelgruppen
schritte	Die Konfigurator-Schritte in der Reihenfolge der Mappe	Liste
preiseFuer(artikelnummer)	Alle Preiszellen eines Artikels	Preiszeilen
findePreis(artikelnummer, achsenwerte)	Preiszelle über die Achsenwerte. Kein Treffer → bewusst „nichts“, nie ein geschätzter Preis.	Preiszeile oder nichts
parseArtikelnummer(nr)	Zerlegt 30-30-05-0011 in die vier Blöcke	Teile oder „ungültig“
sucheNachPraefix(praefix)	Präfix-Suche wie im ERP	Artikelliste
berater, administratoren, filialen	Vorgefilterte Listen (nur aktive)	Listen
getFiliale(idOderAltId)	Filiale per Nummer oder per früherer Code-ID. Alte Entwürfe tragen noch Kennungen wie cramer-wohnvilla-hamburg; die Zuordnung steht als Spalte „Alt-ID“ in der Mappe.	Filiale

4.3 `src/lib/stammdatenStore.ts` — der veränderbare Arbeitsstand

Rolle: Legt die im Admin-Bereich gemachten Änderungen über den erzeugten Grundstand.

Der Grundgedanke (Kommentar im Code): Gespeichert wird nur die **Abweichung**, nicht der ganze Datenbestand — sonst würden eigene Änderungen in 1.509 kopierten Zeilen unsichtbar.

Funktion	Was sie macht
getArtikelliste() / getPreisliste() / ...	Liefern den Arbeitsstand (Grundstand + Änderungen – Löschungen)
preisSchluessel(zeile)	Erzeugt den eindeutigen Schlüssel einer Preiszeile. Achtung: Artikel + Achsenwerte reichen nicht — Artikel ohne Achsen haben mehrere Zeilen mit identisch leeren Achsen. Deshalb steckt zusätzlich die Zeilen-ID der Ursprungs-Extraktion im Schlüssel.
aendereArtikel(nr, patch)	Ändert Felder. Die Artikelnummer selbst wird ignoriert — sie ist unveränderlich.

Funktion	Was sie macht
legeArtikelAn(neu)	Legt an. Prüft das Nummernmuster TT-PP-GG-NNNN und dass die Nummer frei ist.
loescheArtikel(nr)	Entfernt den Artikel samt seiner Preiszeilen — sonst wären sie unauffindbar.
naechsteFreieNummer(vorlage)	Zählt die laufende Nummer im selben Nummernkreis hoch.
dupliziereArtikel(nr)	Kopiert Artikel + Preiszeilen auf die nächste freie Nummer.
aenderePreiszeile / legePreiszeileAn / loeschePreiszeile	Dasselbe Muster für Preise. legePreiszeileAn verweigert eine Zeile zu einem nicht existierenden Artikel.
loescheFiliale(nr)	Verweigert das Löschen, solange noch Mitarbeiter darauf verweisen — und nennt die Namen .
uebernehmeImport(daten)	Übernimmt eine importierte Excel. Verglichen wird gegen den Grundstand , damit das Ergebnis genau die Abweichungen zur Mappe beschreibt.
setzeAllesZurueck()	Verwirft alle Änderungen.
zaehleAenderungen()	Für die Anzeige „x geändert“.
subscribe(hoerer)	Benachrichtigt alle Interessenten bei jeder Änderung.

src/lib/useStammdaten.ts ist das kleine Bindeglied zur Oberfläche: Es sorgt dafür, dass sich Auswahllisten und Kalkulation **sofort** neu berechnen, wenn im Admin-Bereich etwas geändert wurde.

4.4 src/lib/raster.ts — Zentimeter ↔ Raster

Rolle: Berater und Kunde denken in Zentimetern, die Preisliste ist auf Raster geschlüsselt.

Die beiden Formeln (Konstanten aus Blatt „50 Meta“):

Fronthöhe	$H = R \times 128 \text{ mm} - 3 \text{ mm}$	(in allen Programmen gleich)
Korpushöhe	$H = R \times 128 \text{ mm} + \text{Offset}(\text{Serie})$	Atrium 52 · Velare 37 · Publicum 52 · Refugium 46

Funktion	Was sie macht
cmToMm / mmToCm	Umrechnung
korpusOffsetMm(serieId)	Holt den Serien-Offset aus „50 Meta“
rasterFuerHoehe(hoeheCm, offsetMm)	Rasterstufe, aufgerundet — die Preislisten-Regel „Preis des nächstgrößeren Maßes“
frontRaster(hoeheCm)	Kurzform für Fronten
hoeheFuerRaster(raster, offsetMm)	Die Formel rückwärts
istExaktesRastermass(...)	Liegt die Höhe genau auf einer Stufe?
aufVerfuegbaresRaster(raster, verfuegbar)	Hebt auf die nächste tatsächlich bepreiste Stufe. Die Drehtüren-Tabelle kennt nur 4, 6, 8, 9, 15 und 18 Raster. Liegt die Höhe darüber → „nichts“ → Sondermaß, AV-Prüfung.
loeseRasterAuf(...)	Alles zusammen: rechnerische Stufe, bepreiste Stufe, zugehörige Höhe, „ist Sondermaß“, „wurde angehoben“

Warum ganzzahlige Millimeter? Ein bemerkenswertes Detail, das viel über die Sorgfalt des Codes sagt. Kommazahl-Rechnung im Computer greift an exakten Rastermaßen daneben:

```
(153,3 + 0,3) / 12,8 → 12,00000000000002 → aufgerundet 13 FALSCH (richtig: 12)
```

Über alle Programme und Raster 1–21 sind das **genau fünf** Stellen — an allen übrigen kommt zufällig das Richtige heraus. Im Betrieb wäre jedes Möbel dieser Höhen still eine Rasterstufe zu teuer. Deshalb wird ausschließlich in ganzen Millimetern gerechnet.

4.5 `src/lib/preisAchsen.ts` — die Preistabelle lesen

Rolle: Die Preisliste ist eine Extraktion aus einem gedruckten Katalog. Allein die Breite kommt in **sieben** Schreibweisen vor:

50er · 60er (59,5cm) · 50/60er (49,5/59,5cm)	Nennmaß, teils mit Fertigungsmaß
bis 60cm · -140cm · bis 75cm Breite	Obergrenze
T50-51 · T60-230	Katalog-Code (Nennbreite + Rasterschlüssel)
18R · 21R	Raster in der Breitenspalte
Seite (2,0cm)	Bauteilseite
bis 9Raster (120cm)	Höhenklasse (Beleuchtung)
140cm	nacktes Maß

Funktion	Was sie macht
<code>parseZahl(roh)</code>	Zahl lesen, akzeptiert Punkt und Komma
<code>parseListe(roh)</code>	Zerlegt Aufzählungen. Wichtig: In der Mappe ist das Komma bei TIEFE und PG eine Aufzählung , kein Dezimaltrenner — "25,30" meint die Tiefen 25 und 30.
<code>parseBreite(roh)</code>	Zerlegt einen Breitenwert in seine Bestandteile und errechnet einen einheitlichen Vergleichswert in cm
<code>waehleBreite(werte, gesuchtCm)</code>	Wählt den kleinsten Wert, der die verlangte Breite noch abdeckt . Liegt die Anforderung über dem größten bepreisten Wert → bewusst „nichts“.
<code>achsenwertPasst(zellwert, gesucht, opts)</code>	Vergleicht einen Achsenwert mit einer Anforderung; wahlweise als Aufzählung oder als Zahl ("1.5" ≡ "1,5" ≡ "1.50")

Geprüft: Die Einordnung wurde gegen die 1.184 bereits normalisierten Breiten der Vorgänger-Extraktion abgeglichen — Wert für Wert (`npm run data:test`).

4.6 `src/lib/preisLookup.ts` — Artikelnummer + Maße → Preiszelle

Rolle: Das Herz der Bepreisung.

Funktion	Was sie macht	Gibt zurück
<code>findePreis(anfrage)</code>	Sucht die Preiszelle	„gefunden“ mit Preis, Zeile und aufgelösten Achsen — oder „auf-anfrage“ mit Begründung im Klartext

Funktion	Was sie macht	Gibt zurück
verfuegbareRaster(nr)	Die tatsächlich bepreisten Rasterstufen dieses Artikels — aus den Daten gelesen, nicht hinterlegt	Zahlenliste
verfuegbareBreiten(nr)	Die bepreisten Breitenwerte, bereits eingeordnet	Liste
getArtikelNr(nr)	Artikel per Nummer	Artikel

Der Ablauf in `findePreis` — zwei Stufen:

- 1. Alle Achsen außer BREITE als harte Filter.** Raster, Stil-Linie+PG, Preisgruppe, Tiefe, Variante, Bedingung. Bleibt danach nichts übrig, kommt eine Meldung wie: *„Keine Preiszeile für Drehtür mit RASTER=19, LINIE_PG=Glatt2.“*
- 2. Breiten-Bracket wählen.** Unter den verbliebenen Zeilen wird über `waehleBreite()` das nächstgrößere Maß gesucht. Liegt die Breite darüber: *„Breite 250 cm liegt über dem größten Standardmaß (100 cm) — Sondermaß, AV-Prüfung.“*

Warum das wichtig ist: Die Funktion gibt **nie** einen geratenen Preis zurück. Der Kommentar im Code sagt es direkt: *ein falscher Preis im Kundengespräch ist teurer als ein „auf Anfrage“.*

Zusätzlich liefert das Ergebnis die **aufgelösten Achsen mit Klartext-Bedeutung** mit — daraus baut die Positionsliste ihre Nachweiszeile.

Technisches Detail: Die Suchindizes werden aus dem Arbeitsstand aufgebaut und nur dann neu gebaut, wenn sich dort etwas geändert hat. Solange nichts passiert, kostet der Zugriff einen einzigen Zahlenvergleich.

4.7 `src/config/preisMapping.ts` — die Übersetzungstabelle

Rolle: Enthält **keine Logik**, nur Zuordnungen: Welcher Konfigurator-Begriff adressiert welche Artikelnummer? Ein bewusst durchgehaltenes Prinzip steht als Kasten im Code:

In `lib/kalkulation.ts` steht **kein** `if (serieId === 'refugium')`. Alles Serienspezifische gehört hierher.

Eintrag	Inhalt
<code>serienRegeln</code>	Je Serie: Konfigurationsart (BAUTEIL / MODELL / FLÄCHE) und die Artikelnummern für Korpus, Mittelseite, Außenset
<code>getSerienRegel(id)</code>	Holt die Regel
<code>frontLookups</code>	Front-Typ → Artikelnummer, z. B. drehtuer → 20-20-05-0001
<code>liniePgAchsenwert(stillLinie, pg)</code>	Übersetzt Stil-Linie + Preisgruppe in den kombinierten Achsenwert der Preisliste: Glatt + PG2 → "Glatt2"; Curve + PG1 → "CurvePG1", sonst "CurvePG2-4"; Line → "Line"; Glossy/Less → "GlossyLess". Schiene/Classic/Edge haben keine eigene Zeile und werden wie „Glatt“ bepreist.
<code>schubRasterFuerHoehe(hoeheCm)</code>	Schübe sind nach Schubhöhe geschlüsselt: ≤ 12,5 cm → 1 R · ≤ 18,9 cm → 1,5 R · sonst 2 R
<code>ausstattungLookups</code>	Ausstattungs-Option → Artikelnummer (22 Zuordnungen)

Eintrag	Inhalt
containerLookups	Container-Variante → Artikelnummer (Conero A–F, Craft A–C, Basis-Container über Rasterstufe)
ausstattungOhnePreis	Optionen, die bewusst eine Position „auf Anfrage“ erzeugen (Revisionsklappe, Rückwandausschnitt)
prozentZuschlaege	Zuschläge, deren Prozentsatz im Preisblatt steht (Sichrückwand, Raumteiler, wandhängend)
leitePreisgruppeAb(gruppe, option)	Rückfall-Ableitung der Preisgruppe für frei eingegebene Oberflächen

Der aktuelle Ausbaustand der Serienregeln:

Serie	Art	Korpus	Mittelseite	Außenset
Refugium	BAUTEIL	10-10-05-0003	10-10-10-0001	10-10-15-0001
Atrium	BAUTEIL	—	—	—
Velare	BAUTEIL	—	—	—
Publicum	BAUTEIL	—	—	—
Cavum	BAUTEIL	—	—	—
Supersonus	BAUTEIL	—	—	—
Porticus	MODELL	—	—	—
Tavolo	FLÄCHE	—	—	—
Arcum	FLÄCHE	—	—	—

[LÜCKE] Nur **Refugium** ist vollständig hinterlegt. Für die übrigen Serien meldet die Engine ausdrücklich: „Für die Serie atrium sind noch keine Bauteil-Zuordnungen hinterlegt. Die Kalkulation bleibt unvollständig.“ — statt still 0 € zu liefern. Die **Fronten** und die **Innenausstattung** werden dagegen serienübergreifend bepreist, weil frontLookups und ausstattungLookups nicht serienabhängig sind.

4.8 `src/lib/kalkulation.ts` — die Kalkulations-Engine

Rolle: Aus einem Entwurf wird eine bepreiste Positionsliste. Einstiegspunkt ist eine einzige Funktion: `berechneEntwurf(draft)`.

Die sieben Stufen

Stufe	Was passiert
1 · Normalisieren	cm-Maße werden zu Rasterstufen und Breiten-Brackets (aufgerundet)
2 · Ableiten	Mittelseiten und Außenset ergänzen — der Berater wählt sie nie, sie folgen aus dem Aufbau
3 · Prüfen	Plausibilität, Sondermaße, fehlende Angaben
4 · Bepreisen	Je Position ein Lookup über die Artikelnummer

Stufe	Was passiert
5 · Möbelpreis	Summe aller Bauteil-Positionen
6 · Zuschläge	Gestuft: erst % auf den Möbelpreis, dann % auf die Auftragssumme
7 · Ergebnis	Positionen + Summen + Meldungen + Vollständigkeitsstatus

Die drei Prinzipien

1. **Nie raten.** Fehlt eine Preiszeile, entsteht eine Position mit Status „auf Anfrage“. Sie verschwindet nicht und wird nicht geschätzt. Solange eine existiert, ist das Ergebnis `vollstaendig = false`.
2. **Ableitungen begründen.** Jede automatisch ergänzte Position trägt einen Klartext-Hinweis: „*Automatisch ergänzt: 4 Segmente erfordern 3 Mittelseite(n).*“
3. **Jeder Preis ist rückverfolgbar.** Artikelnummer, Achsenwerte A1–A5 und Seite der Preisliste hängen an der Position.

Die Funktionen

Funktion	Was sie macht
<code>berechneEntwurf(draft)</code>	Einstieg. Holt die Serienregel, baut den Kontext, ruft die drei Positions-Bauer, summiert, ergänzt Zuschläge, bewertet die Vollständigkeit.
<code>bauePosition(eingabe)</code>	Der Kern. Eine Position bepreisen: Artikelstamm holen, <code>findePreis()</code> fragen, Ergebnis in eine Positionszeile übersetzen. Bei „auf Anfrage“ wird die Begründung mitgeführt; bei aufgerundeter Breite der Hinweis „ <i>Sondermaß: bepreist mit dem nächstgrößeren Maß 100er.</i> “
<code>leseKorpusKontext(...)</code>	Ermittelt Höhe, Tiefe und die Breiten der Segmente. Löst die Höhe in eine bepreiste Rasterstufe auf. Warnt, wenn die Summe der Segmentbreiten um mehr als 0,5 cm von der erfassten Gesamtbreite abweicht.
<code>baueKorpusPositionen(...)</code>	Je Segment ein Korpus. Dann abgeleitet : Mittelseiten = Segmente – 1; Außenset, sofern nicht ausdrücklich „keine“ gewählt wurde. Fehlt für das Außenset die Preisgruppe, kommt ein Fehler , kein stiller 0-€-Posten.
<code>baueFrontPositionen(...)</code>	Je Front-Element eine Position. Ermittelt Breite (eigene oder Segmentbreite), Preisgruppe aus dem Materialfeld, Stil-Linie+PG als Achsenwert, Rasterstufe aus der Höhe. „Offen (Regal)“ erzeugt bewusst keine Position. Ist ein Griff gewählt, entsteht dafür eine eigene Position.
<code>griffArtikelnummer(griffId)</code>	Übersetzt die Griff-Kennung des Configurators (nr127) in eine Artikelnummer — über die Nummer im Namen , damit ein neuer Griff in der Mappe ohne Code-Änderung gefunden wird.
<code>baueAusstattungsPositionen(...)</code>	Je Ausstattungs-Element eine Position. Container werden über ihre Variante aufgelöst. Optionen ohne hinterlegte Zuordnung erzeugen eine Position „auf Anfrage“ mit dem Hinweis, wo die Zuordnung fehlt.
<code>baueZuschlaege(...)</code>	Zweistufig: erst Prozent auf den Möbelpreis (Satz kommt aus dem Preisblatt , nicht aus dem Code), dann Montage (10 %) und Lieferung regional (3 %) auf die Zwischensumme.
<code>zuschlagsProzent(nr)</code>	Holt den Prozentsatz als Preis eines Zuschlag-Artikels

Das Ergebnis enthält: Positionen, Zuschläge, Möbelpreis, Gesamt, Meldungen (Fehler / Warnung / Info), Anzahl offener Positionen, `vollstaendig` (ja/nein), Gültigkeit („06.2026“) und Währung.

4.9 Die Regel-Dateien der Oberfläche

Datei	Funktion	Was sie macht
config/korpus.ts	getVisibleKorpusAreas(serie, modus)	Welche Korpus-Bereiche sind sichtbar? „Innen“ nur bei Velare/Refugium; „Abdeckplatte“ weg bei Serien ohne; „getrennt“ ersetzt „Außen“ durch links/rechts.
lib/materialRules.ts	resolvePriceGroup(gruppe, option)	Preisgruppe zuweisen — Option schlägt Gruppe
	isMaterialSelectionComplete(auswahl)	Vollständig, wenn: „Keine ...“ gewählt oder „anders“ mit Text oder Gruppe + Option
lib/korpusValidation.ts	getIncompleteKorpusAreas(...)	Liefert die IDs der noch offenen Pflichtbereiche
lib/dimensionsValidation.ts	validateDimensions(...)	Prüft H/B/T auf positive Zahlen und Segmente auf 1–12
lib/draftValidation.ts	validateDraftForm(...)	Kundenname und Filiale sind Pflicht — Auftrags-/Artikelnummer nicht
lib/frontsValidation.ts	isFrontElementValid(el)	Kennzeichnung gesetzt, Höhengrenze eingehalten, Stil-Linie gewählt, jedes Materialfeld gefüllt
	exceedsMaxHeight(el)	Schreibklappe und Stauraumklappe max. 45 cm
	getFrontsIssues(fronts)	Erzeugt die Klartext-Liste offener Punkte unter den Buttons
	isFrontsComplete(fronts)	Sonderfall zweiläufige Schiebetür: leere Segmente sind dann zulässig, dafür ist die Türanzahl Pflicht
lib/frontsHelpers.ts	syncColumns(fronts, segmente)	Gleicht die Spaltenzahl an die Segmente an, ohne Inhalte zu verlieren
	canAddFrontType(fronts, typ)	Die Exklusivitäts-Regel der zweiläufigen Schiebetür
	schiebetuerAnzahlOptions(n)	2 Korpi → 2 · 3 → 3 · 4 → 2 oder 4
	eligibleEquipmentFrontTypes(spalte)	Hinter welchen Fronten dieses Segments ist Ausstattung möglich?
	copyableFrontValues(front)	Die kopierbaren Werte für „Werte von Front 1 übernehmen“
	makeId(praefix)	Erzeugt eine eindeutige Kennung
	resolveHeightCm/DepthCm/KorpusBreiteCm	Modus (18R / 60 / 50er / „anders“) → Zahl in cm
lib/korpusMass.ts	computeKorpusKoordinaten(g)	Linke Kante je Korpus — Basis für spätere 2D/3D-Darstellung
	deriveDimensions(g)	Strukturierte Grunddaten → einfache H/B/T + Segmentzahl
	isSondertiefeDepth(draft)	Tiefe < 60 cm? Steuert die Ausstattungs-Sperre
	describeKorpusGrunddatenZeilen(g)	Klartext-Zeilen für Zusammenfassung und PDF — identisch, aus einer Quelle
config/equipment.ts	getEquipmentOption(id)	Katalog-Option nachschlagen
	defaultSelectedEquipmentIds()	Die vorausgewählten Essentials

Datei	Funktion	Was sie macht
	<code>isEquipmentAvailableInSondertiefe(id)</code>	Nur der einfache Einlegeboden

4.10 Zustand und Speicherung

`src/context/DraftContext.tsx` — der Entwurf als zentrale Wahrheit.

Funktion	Was sie macht
<code>startNewDraft(berater)</code>	Legt einen Entwurf mit automatischer Nummer an
<code>updateDraft(patch)</code>	Ergänzt Felder — von jedem Schritt aufgerufen
<code>saveDraft()</code> / <code>finalizeDraft()</code>	Speichern bzw. als abgeschlossen markieren
<code>loadDraft(id)</code> / <code>deleteDraft(id)</code>	Laden / löschen
<code>duplicateDraft(id)</code>	Vollständige Kopie als Variante : neue Entwurfsnummer, Verweis <code>variantOf</code> auf das Original, Auftrags-/Artikelnummer und Abschluss geleert

Automatisches Speichern: Sobald ein Entwurf identifizierbaren Inhalt hat (Kunde, Auftrag, Artikelnummer oder Produktgruppe), wird er automatisch in die Liste übernommen — er kann nicht verloren gehen.

Speicherort: Browser-Speicher (`localStorage`), Schlüssel `cramer-planer.draft.current.v2` und `cramer-planer.drafts.v2`. Alle Entwurfsdaten sind reines JSON — also direkt geeignet für eine spätere Datenbank oder Schnittstelle.

`src/context/AuthContext.tsx` — Anmeldung, Rollen (Berater/Administrator), Benutzerverwaltung, Wartungsmodus. Passwörter werden als SHA-256-Hash abgelegt (`lib/passwort.ts`), Klartext wird nirgends gespeichert.

4.11 Ausgabe und Schnittstellen

Datei	Funktion	Was sie macht
<code>lib/generatePdf.ts</code>	<code>buildPdf(draft)</code> / <code>downloadPdf(draft)</code>	Erzeugt das AV-PDF im Browser (Bibliothek <code>jspdf</code>). Seite 1: alle Daten; Seite 2: die gescannte Skizze.
<code>lib/scanBridge.ts</code>	<code>getScanHost</code> / <code>buildMobileScanUrl</code> / <code>postScan</code> / <code>fetchScan</code>	Die Smartphone-Brücke: Der Laptop zeigt einen QR-Code, das Smartphone öffnet <code>/scan/<Entwurfsnummer></code> , fotografiert, das Bild wird optimiert und an den Laptop übertragen.
<code>lib/stammdatenExport.ts</code>	<code>exportiereXlsx</code> / <code>importiereXlsx</code>	Die Excel-Runde. Spaltennamen identisch zur Mappe; Import ordnet über Überschriften zu.
<code>lib/xlsxBrowser.ts</code>	<code>leseXlsx</code> / <code>schreibeXlsx</code>	Liest und schreibt Excel-Dateien direkt im Browser (Bibliothek <code>fflate</code>)
<code>scripts/build-stammdaten.js</code>	<code>main()</code>	Excel + Markdown → getypes Modul, mit Kreuzprüfung

Datei	Funktion	Was sie macht
scripts/check-lookup.js	—	Selbsttest der Modus- und Lookup-Logik gegen die echten Daten
scripts/check-kalkulation.js	—	Vergleicht die Kalkulation gegen die Sollwerte des Vorgänger-Tools — 22 Sollwerte, alle erreicht

[LÜCKE] Die Scan-Brücke ist ein **Zwischenspeicher im Arbeitsspeicher**. Im Entwicklungs-Server (`vite.config.ts`) und in der Vercel-Funktion (`api/scan.ts`) ist das explizit als Platzhalter markiert. Produktiv braucht es einen geteilten Speicher oder eine echte Push-Verbindung.

4.12 Reste aus der Vorgänger-Fassung

Der Vollständigkeit halber, weil es in der Dateiliste auffällt:

- `src/data/priceList.json` (1 MB) und `src/config/pricing.ts` sind die **alte** Preisquelle. Die zugehörige Suchfunktion `findPrice()` in `src/lib/pricing.ts` wird **nirgends mehr aufgerufen**. Aus dieser Datei werden nur noch die beiden Formatierungs-Hilfen `formatEuro` und `formatVkPreis` benutzt. Die Kalkulation läuft vollständig über das Excel-Preisblatt.
- `src/config/handles.ts` enthält je Griff ein Feld `priceEur`. Dieses Feld wird **nicht mehr verwendet** — der Griffpreis kommt aus dem Artikelstamm. Der Katalog dient nur noch der Anzeige-Bezeichnung.
- `CONTAINER_RAUCHGLAS_ARTIKEL` in `preisMapping.ts` ist definiert, wird aber in der Kalkulation noch nicht ausgewertet — der Aufpreis „Deckplatte Rauchglas“ wird erfasst, aber nicht bepreist.

[EMPFEHLUNG] Diese drei Reste vor der Produktivsetzung entfernen bzw. anschließen. Sie schaden heute nichts, sind aber Stolperstellen für jeden, der später am Code arbeitet.

KAPITEL 5

Zusammenspiel von Oberfläche, Daten und Logik

5.1 Beispiel: Der Benutzer klickt auf „Griff“

Was der Benutzer sieht: In der Front-Karte einer Drehtür mit Stil-Linie „Glatt“ stehen zwei Häkchen: „PTO (Push-to-Open)“ und „Griff“. Er setzt das Häkchen bei „Griff“. Darunter erscheint ein Dropdown „Griff-Auswahl“ mit 15 Einträgen (bzw. 11 bei Glossy/Less).

Was technisch passiert — Schritt für Schritt:

- 1. Welche Funktion wird ausgelöst?** Das `onChange` des Häkchens in `FrontElementCard.tsx` ruft `onChange({ griff: true, pto: false })`. Das Feld `pto` wird dabei **zwangsweise** auf `false` gesetzt — PTO und Griff schließen sich aus. Zusätzlich ist das jeweils andere Häkchen deaktiviert, solange eines gesetzt ist. Die Regel ist also doppelt abgesichert.
- 2. Woher weiß das System, dass überhaupt Griffe angezeigt werden?** Nur wenn die gewählte Stil-Linie im Katalog das Merkmal `handleOptions: true` trägt. In `frontCatalog.ts` ist das bei **Glatt, Glossy und Less** der Fall — nicht bei `Line, 107, Curve, Schiene, Classic` oder `Edge`. Diese Linien sind grifflos bzw. haben integrierte Griffprofile.
- 3. Woher kommt die Liste der Griffe?** `getAvailableHandles(stilLinie)` aus `config/handles.ts` liefert 15 Griffe. Bei Glossy oder Less werden vier davon herausgefiltert: Nr. 103, 125, 126 und 128 (Konstante `GLOSSY_LESS_EXCLUDED_HANDLE_IDS`). Unter dem Dropdown steht dann der Hinweis *„Bei Glossy/Less nicht möglich: Griff Nr. 103, 125, 126, 128.“*
- 4. Was passiert bei einem späteren Stil-Wechsel?** `pickStyleLine()` prüft, ob der bereits gewählte Griff in der neuen Linie noch zulässig ist — wenn nicht, wird er verworfen. Der Berater kann also keine unzulässige Kombination „durchschleppen“, indem er erst den Griff und dann den Stil wählt.
- 5. Wo kommt der Preis her?** Erst am Ende, in der Kalkulation. `baueFrontPositionen()` sieht, dass `griff` und `griffId` gesetzt sind, ruft `griffArtikelnummer('nr127')` auf. Diese Funktion zieht die Zahl aus der Kennung (127) und sucht im **Artikelstamm** unter allen Artikeln der Gruppe GRIFF (Nummernkreis 30-30-05-) den, dessen Bezeichnung „Nr. 127“ enthält → 30-30-05-0011. Damit fragt sie `findePreis()` → 60,00 € (Preisliste Seite 3).
- 6. Was steht danach in der Positionsliste?**

Griff Nr. 127 Griffleiste (200 mm) „D1“	1x	60,00 €
30-30-05-0011 · GRF-011 · BESCHLAG · FRONTAUSSTATTUNG · Stück · Preisliste S. 3		

Bemerkenswert: Die Verbindung Konfigurator → Artikelstamm läuft hier bewusst **über die Nummer im Namen**, nicht über eine fest verdrahtete Tabelle. Kommt ein neuer Griff „Nr. 130“ in die Mappe, findet ihn die Kalkulation ohne Code-Änderung — sobald er in der Auswahlliste steht. Und genau da liegt die verbleibende Lücke: In der Auswahlliste steht er erst nach einer Ergänzung in `handles.ts`.

5.2 Beispiel: Der Benutzer wählt eine Serie

Was der Benutzer sieht: Er klickt „Kleiderschränke“, darunter erscheint genau eine Serie: Refugium. Er klickt sie an, darunter steht: *„Serien-Kürzel R · 109 von 190 Artikeln freigegeben, verteilt auf 7 Schritte ...“*

Technisch:

1. `productGroups` aus `productCatalog.ts` liefert die Kacheln.
2. `selectGroup()` setzt die Gruppe und **löscht die Serie** — nie eine gruppenfremde Serie.
3. `artikelFuerSerie('refugium')` läuft durch alle 190 Artikel des Arbeitsstands und behält die, deren Status aktiv ist **und** deren Modus ein R enthält.
4. `dropdownsFuerSchritt(code, 'refugium')` prüft je Schritt, welche Artikelgruppen mindestens einen freigegebenen Artikel haben.
5. Weiter geht es nur, wenn `getSeries(gruppe, serie)` einen Treffer liefert.

Der Beleg im Betrieb: Diese Zahl ist der sichtbare Nachweis, dass die Modus-Filterung tatsächlich greift. Ändert jemand im Admin-Bereich den Modus eines Artikels, ändert sich die Zahl sofort — ohne Neuladen (über `useStammdaten()`).

5.3 Beispiel: Der Benutzer wählt ein Material

Was der Benutzer sieht: Chips „Decoboard · Mattlack · Furnier · Gläser · Xtreme Plus · anders“, danach ein Dropdown mit Farben, danach dezent „Preisgruppe 2“.

Technisch:

1. Welche Chips erscheinen, sagt die Bereichs-Konfiguration: `korpus.ts` für den Korpus (Innen erlaubt nur Decoboard!), `frontCatalog.ts` für Fronten (Glossy/Less erlauben nur Glas, Curve erlaubt kein Glas, Line nur Glas/Mattlack/Furnier).
2. `MaterialSelect` lädt die Gruppen aus der zentralen Farbmatrix und zeigt die Optionen.
3. `excludeOptionIds` kann einzelne Optionen ausschließen — bei der Abdeckplatte werden alle RauchglasVarianten entfernt. Diese Liste wird **dynamisch aus der Matrix** ermittelt (Treffer auf „rauchglas“ in ID oder Bezeichnung), damit auch künftige Rauchglas-Varianten automatisch erfasst sind.
4. `resolvePriceGroup()` setzt die Preisgruppe.
5. Trägt die Option das Merkmal `requiresFreeText`, klappt ein zusätzliches Feld auf — bei „NCS (Sonderfarbe)“ etwa „NCS-Farbbezeichnung“.

5.4 Beispiel: „Ausstattung hinter der Front“

Was der Benutzer sieht: Ein aufklappbarer Bereich unter den Fronten eines Segments, darin nur die Optionen, die er in Schritt 6 angehakt hat.

Technisch — eine dreifache Filterung:

```
Alle Optionen des Katalogs (equipment.ts)
  ↓ Filter 1: nur die in Schritt 6 vorausgewählten
  ↓ Filter 2: nur die, deren erlaubte Front-Typen im Segment vorkommen
  ↓           (Innenspiegel nur bei Drehtür)
  ↓ Filter 3: bei Sondertiefe (< 60 cm) nur Einlegeböden
Angebotene Optionen
```

Und der Bereich erscheint überhaupt nur, wenn das Segment eine Drehtür, eine zweiläufige Schiebetür oder ein offenes Fach enthält. Hinter einer Schublade wird keine Kleiderstange angeboten.

KAPITEL 6

Regeln und Fehlervermeidung

Der Planer verhindert unzulässige Kombinationen auf **sechs** unterschiedlichen Wegen. Wichtig für das Kundengespräch: Diese Wege sind bewusst verschieden, weil sie verschiedene Zwecke haben.

6.1 Filterregeln — was gar nicht erst erscheint

Das stärkste Mittel: Eine unzulässige Option ist nicht auswählbar, weil sie nicht angezeigt wird.

Regel	Wo umgesetzt
Nur Serien der gewählten Produktfamilie	<code>productCatalog.ts · getSeries()</code>
Nur Artikel, deren Modus die Serie enthält	<code>lib/stammdaten.ts · findeArtikel()</code>
Nur Artikel mit Status aktiv	<code>lib/stammdaten.ts · findeArtikel()</code>
Leere Dropdowns verschwinden ganz	<code>lib/stammdaten.ts · dropdownsFuerSchritt()</code>
„Innen“ nur bei Velare/Refugium	<code>config/korpus.ts · getVisibleKorpusAreas()</code>
Keine Abdeckplatte bei Refugium	<code>productCatalog.ts (hasAbdeckplatte: false)</code>
Keine Sichtrückwand bei Refugium	<code>productCatalog.ts (hasSichtRueckwand: false)</code>
Zweiläufige Schiebetür nur bei Refugium	<code>frontCatalog.ts (refugiumOnly)</code>
Nur zulässige Materialgruppen je Feld	<code>korpus.ts / frontCatalog.ts</code>
Rauchglas nicht bei der Abdeckplatte	<code>Korpus.tsx + RAUCHGLAS_OPTION_IDS</code>
Bei Glossy/Less vier Griffe weniger	<code>handles.ts · getAvailableHandles()</code>
Bei Sondertiefe nur Einlegeböden	<code>equipment.ts · isEquipmentAvailableInSondertiefe()</code>
Schritt „Ausstattung“ nur bei Refugium	<code>workflow.ts · visibleWorkflowSteps()</code>

6.2 Kompatibilitätsregeln — was sich gegenseitig ausschließt

Regel	Wo umgesetzt
PTO oder Griff — nie beides	<code>FrontElementCard.tsx</code> (Zwangssetzung und Deaktivierung)
Zweiläufige Schiebetür ist exklusiv — dann keine weitere Front im ganzen Schrank	<code>frontsHelpers.ts · canAddFrontType()</code>
Türanzahl abhängig von der Korpuszahl	<code>frontsHelpers.ts · schiebetuerAnzahlOptions()</code>
Stil-Wechsel verwirft unzulässig gewordenen Griff	<code>FrontElementCard.tsx · pickStyleLine()</code>
Bei 107/Curve entfällt der Griffleisten-Freitext, wenn Decoboard oder Xtreme Plus gewählt ist	<code>frontCatalog.ts · hideWhenSiblingGroupIn</code>
Ausstattung nur hinter Drehtür / zweiläufiger Schiebetür / offenem Fach	<code>equipment.ts · EQUIPMENT_ELIGIBLE_FRONT_TYPES</code>

Regel	Wo umgesetzt
Innenspiegel nur bei Drehtür	<code>equipment.ts (frontTypes: ['drehtuer'])</code>
Filiale nicht löscher, solange Mitarbeiter darauf verweisen	<code>stammdatenStore.ts · loescheFiliale()</code>
Artikel löschen entfernt seine Preiszeilen mit	<code>stammdatenStore.ts · loescheArtikel()</code>
Preiszeile ohne Artikel nicht anlegbar	<code>stammdatenStore.ts · legePreiszeileAn()</code>

6.3 Pflichtfelder — erzwungene Progression

Der „Weiter“-Knopf ist gesperrt, solange etwas fehlt. Zusätzlich wird jede Seite beim Betreten geprüft: Wer per Adresszeile zu `/fronts` springt, ohne den Korpus ausgefüllt zu haben, wird automatisch zurückgeschickt.

Schritt	Pflicht
Entwurf	Kundenname, Filiale
Produkt	Produktgruppe und Serie
Korpus	Alle sichtbaren Bereiche vollständig (Abdeckplatte auch durch „Keine Abdeckplatte“ erfüllbar)
Maße	Höhe, Breite, Tiefe > 0; Segmente 1–12; bei Refugium: jede Korpusbreite gültig
Fronten	Je Segment mindestens ein Bauteil; je Bauteil Kennzeichnung, Stil-Linie und jedes Materialfeld
AV-Übergabe	Auftragsnummer und Artikelnummer — erst hier, nicht vorher

Bei den Fronten wird zusätzlich eine **Klartext-Liste** offener Punkte angezeigt: „Front-Typ 2 · D3: Material auswählen.“

6.4 Plausibilitätsprüfungen — Hinweise statt Sperren

Diese Regeln verhindern nichts, sie machen sichtbar:

Prüfung	Meldung
Segmentbreiten ≠ erfasste Gesamtbreite (> 0,5 cm)	Warnung mit beiden Zahlen und der Differenz
Höhe ist kein exaktes Rastermaß	Info: „Höhe 200 cm ist kein Rastermaß (rechnerisch 16 R). Bepreist mit 18 R ≈ 235 cm.“
Höhe über der größten bepreisten Stufe	Fehler: Sondermaß, AV-Prüfung
Breite über dem größten Standardmaß	Position auf Anfrage mit Nennung des größten Maßes
Schreib-/Stauraumklappe über 45 cm	Rote Warnung an der Karte und Element gilt als ungültig
Keine Korpus-Grunddaten vorhanden	Info: „Gesamtbreite wurde gleichmäßig auf n Segmente verteilt.“
Kleiderstange im Korpus-Innenausbau ohne Menge	Info: bitte je Segment erfassen
Serie ohne Bauteil-Zuordnung	Warnung: Kalkulation bleibt unvollständig
Preisgruppe für das Außenset fehlt	Fehler

6.5 Preislogik — nie raten

Das strengste Prinzip des Systems:

- Findet der Lookup keine Preiszelle, entsteht eine Position mit Status „**auf Anfrage**“ und einer Begründung im Klartext. Sie verschwindet nicht aus der Liste.
- Solange auch nur eine offene Position existiert, steht das Ergebnis auf `vollstaendig = false`, und die Oberfläche zeigt statt „vollständig kalkuliert“ den Text „n Position(en) offen“.
- Unter dem Knopf „Berechneten Preis übernehmen“ steht dann: *„Solange Positionen offen sind, ist der berechnete Preis nicht verbindlich.“*
- Individuelle Maße werden **immer nach oben** auf das nächste bepreiste Maß gehoben, nie nach unten — und der Hinweis steht an der Position.
- Der **verbindliche Preis ist immer der manuell eingetragene VK-Preis**. Die Kalkulation liefert einen begründeten Vorschlag; die Entscheidung bleibt beim Berater.

6.6 Gesperrte Artikel

Zwei Sperren, unabhängig voneinander:

1. `findeArtikel()` liefert Artikel mit Status `gesperrt` oder `entwurf` **gar nicht** aus — sie erscheinen in keiner Auswahlliste.
2. `findePreis()` prüft den Status noch einmal und antwortet bei einem gesperrten Artikel mit *„Artikel 30-30-05-0011 ist gesperrt.“* statt mit einem Preis.

Der zweite Punkt ist wichtig für **bestehende Entwürfe**: Wird ein Artikel gesperrt, nachdem er in einem Entwurf verwendet wurde, verschwindet er dort nicht still — er wird als Position „auf Anfrage“ mit Begründung ausgewiesen.

6.7 Prüfungen beim Erzeugen der Stammdaten

Vor jedem Programmstart prüft `build-stammdaten.js`:

- Excel gegen Markdown: Stimmen Teilearten, Produktgruppen und Artikelgruppen überein?
- Verweist jeder Artikel auf existierende Produktgruppe, Artikelgruppe, Teileart, Preislogik und Achsen?
- Gibt der Modus jedes Artikels mindestens eine Serie frei? („... *sonst erscheint der Artikel nirgends.*“)
- Verweisen alle Preiszeilen auf existierende Artikel?
- Verweisen alle Mitarbeiter auf existierende Filialen?

Alles wird als Hinweis ausgegeben — das Skript bricht nicht ab, sondern macht sichtbar.

KAPITEL 7

Ein vollständiges Beispiel

Ich spiele die Konfiguration durch, die auch der Selbsttest `npm run kalk:test` verwendet — mit **echten Zahlen aus den echten Daten**. Der Test läuft und erreicht alle 22 Sollwerte.

Das Möbel: Refugium-Kleiderschrank, 250 × 200 × 60 cm, 4 Korpus-Segmente.

Schritt 1 — Entwurf anlegen

Der Berater tut	Ausgelöste Funktion	Ergebnis
Kunde und Filiale eintragen	<code>validateDraftForm()</code>	Beide Pflichtfelder erfüllt
„Entwurf anlegen“	<code>startNewDraft() → generateEntwurfsnummer()</code>	CRAMER-2026-AB-0001

Schritt 2 — Produkt wählen

Der Berater tut	Ausgelöste Funktion	Gesucht wird	Ergebnis
Kachel „Kleiderschränke“	<code>selectGroup()</code>	<code>productCatalog.ts</code>	Nur Refugium wird angeboten
Serie „Refugium“	<code>artikelFuerSerie('refugium')</code>	190 Artikel, Feld Modus + Status	109 Artikel freigegeben, 25 Artikelgruppen

Greifende Regeln: Modus-Filter (R), Status-Filter (aktiv), Gruppenzugehörigkeit.

Schritt 3 — Korpus (Material)

Der Berater tut	Funktion	Datenquelle	Ergebnis
Bereich „a. Innen“ → Decoboard → Eiche Milano	<code>resolvePriceGroup()</code>	<code>materialMatrix.ts</code>	PG1 automatisch gesetzt
Bereich „b. Außen“ → Matlack → Beigegrau	<code>resolvePriceGroup()</code>	<code>materialMatrix.ts</code>	PG2 automatisch gesetzt
Abdeckplatte	<code>getVisibleKorpusAreas()</code>	<code>productCatalog.ts</code>	Erscheint gar nicht — Refugium hat keine
Sicht-Rückwand	<code>series.hasSichtRueckwand</code>	<code>productCatalog.ts</code>	Erscheint gar nicht

Für „Innen“ sind nur Decoboard-Farben wählbar (`korpus.ts: materialGroupIds: ['decoboard']`) — passend zum Kommentar in `preisMapping.ts`: „Refugium-Korpi sind ausschließlich in Decoboard lieferbar.“

Schritt 4 — Maße (Korpus-Grunddaten)

Der Berater tut	Funktion	Ergebnis
Höhe: „anders“ → 200 cm	resolveHeightCm()	200
Tiefe: Standard 60 cm	resolveDepthCm()	60
4 Korpusse: 3× 60er, 1× 100er	resolveKorpusBreiteCm()	[60, 60, 60, 100]
„Weiter“	deriveDimensions() + syncColumns()	4 leere Front-Spalten entstehen

Die Rasterauflösung — hier wird es interessant:

```

korpusOffsetMm('refugium')      → 46 mm          (aus Blatt „50 Meta“)
rasterFuerHoehe(200, 46)        → (2000 - 46) ÷ 128 = 15,3 → aufgerundet 16 R
verfuegbareRaster('10-10-05-0003') → [18, 21]      (aus den echten Preiszeilen)
aufVerfuegbaresRaster(16, [18,21]) → 18 R
hoeheFuerRaster(18, 46)         → 235 cm

```

Meldung an den Berater: „Höhe 200 cm ist kein Rastermaß (rechnerisch 16 R). Bepreist mit 18 R ≈ 235 cm.“

Das ist genau richtig: Die Preisliste kennt für den Refugium-Korpus nur 18 und 21 Raster. Ein 200-cm-Schrank kostet so viel wie ein 235-cm-Schrank. Der Berater erfährt das **im Gespräch**, nicht später von der AV.

Schritt 5 — Ausstattung-Vorauswahl

Der Berater tut	Funktion	Ergebnis
Seite öffnen	defaultSelectedEquipmentIds()	Einlegeboden + Einlegeboden inkl. Kleiderstange sind vorausgewählt
Zusätzlich: LED-Band, Innenschublade, Innenspiegel	toggle()	Auswahl gespeichert
(Tiefe = 60 cm)	isSondertiefeDepth()	false → keine Sperre

Schritt 6 — Fronten

Der Berater legt an: Segment 1 eine Drehtür „D1“, Segment 2 eine Drehtür „D2“, Segment 3 zwei Schubladen „S1“/„S2“, Segment 4 eine Drehtür „D3“. An „D1“ wählt er den Griff Nr. 127.

Für „D1“ im Detail:

Der Berater tut	Funktion	Ergebnis
„+ Drehtür“	canAddFrontType()	Erlaubt (keine zweiläufige Schiebetür vorhanden)
Kennzeichnung „D1“	isFrontElementValid()	Pflicht erfüllt
Breite 60, Höhe 230	—	Gespeichert
Stil-Linie „Glatt“	getStyleLine()	Materialfeld + Griff-Optionen erscheinen
Material → Mattlack → Beigegrau	resolvePriceGroup()	PG2
Häkchen „Griff“	—	pto wird zwangsweise false; Dropdown erscheint

Der Berater tut	Funktion	Ergebnis
Griff „Nr. 127 Griffleiste“	getAvailableHandles('gla tt')	Alle 15 Griffe wählbar

Schritt 7 — Die Kalkulation im Detail

Jetzt läuft `berechneEntwurf(draft)`. Ich zeige jede Stufe.

Stufe 1–3: Kontext

```
leseKorpusKontext()
  Höhe 200 cm · Tiefe 60 cm · Breiten [60, 60, 60, 100]
  Rasterstufe → 18 R (mit Hinweis, siehe oben)
  Serienregel: getSerienRegel('refugium') → Korpus 10-10-05-0003, Mittelseite 10-10-10-0001,
  Außenset 10-10-15-0001
  Serien-Infos: „Refugium hat serienmäßig keine Abdeckplatte (Programmvergleich S. 6).“
  „Die 64er Lochreihe ist Serienstandard und damit keine Preisposition.“
```

Stufe 4a: Korpus und die abgeleiteten Teile

Position	Artikel	Achsen	Menge	Einzel	Gesamt
Korpus 1	10-10-05-0003	A1 = 60er · A2 = 18	1	258,00	258,00
Korpus 2	10-10-05-0003	A1 = 60er · A2 = 18	1	258,00	258,00
Korpus 3	10-10-05-0003	A1 = 60er · A2 = 18	1	258,00	258,00
Korpus 4	10-10-05-0003	A1 = 100er · A2 = 18	1	309,00	309,00
Mittelseite (abgeleitet)	10-10-10-0001	A1 = 18R	3	135,00	405,00
Außenset (abgeleitet)	10-10-15-0001	A1 = 18R · A2 = PG2	1	359,00	359,00

Was hier passiert ist — und was der Berater nie eingeben musste:

- Die **Mittelseiten** hat niemand gewählt. `baueKorpusPositionen()` rechnet: 4 Segmente → 3 Trennwände. Die Position trägt den Hinweis „Automatisch ergänzt: 4 Segmente erfordern 3 Mittelseite(n).“
- Das **Außenset** hat niemand gewählt. Es wird ergänzt, solange nicht ausdrücklich „keine“ gewählt wurde, mit dem Hinweis „Automatisch ergänzt: schließt den Möbelblock seitlich ab. Trägt die Oberfläche des Möbels nach außen — der Korpus selbst ist nur in Decoboard lieferbar.“ Seine Preisgruppe (PG2) kommt vom **Außen**-Material des Korpus.
- Bemerkenswert: Beim Korpus und der Mittelseite steht die Rasterstufe **in der Breitenspalte** („18R“), nicht in einer eigenen Rasterachse. Genau dafür gibt es im Lookup das Merkmal `rasterInBreite`.

Stufe 4b: Fronten

Position	Artikel	Achsen	Einzel
Drehtür „D1“	20-20-05-0001	A1 = T60-230 · A2 = 18 · A3 = Glatt2	348,00
Griff Nr. 127	30-30-05-0011	A1 = 200mm	60,00
Drehtür „D2“	20-20-05-0001	A1 = T60-230 · A2 = 18 · A3 = Glatt2	348,00

Position	Artikel	Achsen	Einzel
Schublade „S1“	20-20-20-0001	Breite · Raster · Linie+PG · Tiefe	346,00
Schublade „S2“	20-20-20-0001	dieselben	346,00
Drehtür „D3“	20-20-05-0001	—	auf Anfrage

Wie 348,00 € zustande kommt — der komplette Weg:

1. frontLookups['drehtuer'] → Artikel 20-20-05-0001
nutztLiniePg: true, nutztRaster: true
2. liniePgAchsenwert('glatt', 'PG2') → "Glatt2" (Regel: Glatt N = PG N)
3. verfuegbareRaster('20-20-05-0001') → [4, 6, 8, 9, 15, 18]
4. loeseRasterAuf(230, -3, [4,6,...,18]) → $(2300 + 3) \div 128 = 17,99 \rightarrow 18$ R (vorhanden)
5. findePreis({ artikel: '20-20-05-0001', breiteCm: 60, raster: 18, liniePg: 'Glatt2' })

Stufe 1 – harte Filter:

- A2 Raster = 18 von 120 Zeilen bleiben 20
A3 LiniePG = Glatt2 davon bleiben 2: T50-230 und T60-230

Stufe 2 – Breiten-Bracket:

- parseBreite("T50-230") → Katalog-Code, Nennbreite 50 cm
parseBreite("T60-230") → Katalog-Code, Nennbreite 60 cm
waehleBreite([50, 60], gesucht 60) → kleinster Wert $\geq 60 \rightarrow$ T60-230

6. Treffer: 348,00 €, Preisliste Seite 7

Warum „D3“ auf Anfrage steht: Segment 4 ist 100 cm breit. Die Drehtüren-Tabelle kennt nur Nennbreiten 50 und 60 cm. waehleBreite() findet keinen Wert ≥ 100 und gibt „nichts“ zurück. Die Meldung lautet:

„Breite 100 cm liegt über dem größten Standardmaß (60 cm) — Sondermaß, AV-Prüfung.“

Das ist **fachlich richtig**: Eine 100 cm breite Drehtür ist kein Standardartikel. Und es ist genau das Verhalten, das den Berater schützt — hier wird nicht hochgerechnet, hier wird gefragt.

Stufe 4c: Innenausstattung

Position	Artikel	Menge	Gesamt
Einlegeboden	40-40-05-0001	4	180,00
Kleiderstange	40-40-10-0002	1	60,00
LED-Band	50-50-05-0006	—	1.000,00
Innenschublade	40-40-15-0001	—	378,00
Innenspiegel	40-40-25-0001	1	130,00

Beim **LED-Band** greift eine Besonderheit: Das Merkmal breiteAusKorpushoehe sagt dem Lookup, dass hier **nicht** die Segmentbreite, sondern die **Korpushöhe** als Breitenwert einzusetzen ist — die Preisliste schlüsselt es nach Höhenklasse („bis 18Raster (235cm)“). Die Position trägt den Hinweis „Bepreist nach Korpushöhenklasse, je Schrankseite.“

Stufe 5–7: Summen

Möbelpreis (17 Positionen, davon 1 auf Anfrage)	5.043,00 €	
Montage (+10 %)	504,30 €	← Satz aus „50 Meta“
Gesamt	5.547,30 €	

Status: 1 Position offen → vollstaendig = false

Meldung: „1 Position(en) ohne Preis – Angebot nur unter Vorbehalt, AV-Prüfung erforderlich.“

Das ist der verifizierte Sollwert. `npm run kalk:test` vergleicht dieses Ergebnis Position für Position mit der Ausgabe des Vorgänger-Tools und meldet: „Alle 22 Sollwerte erreicht — die neue Engine rechnet identisch.“

So sieht eine Position auf dem Bildschirm aus

```
Drehtür „D1“                                1x      348,00 €
20-20-05-0001 · DRT-001 · FRONT · FRONT · Stück · Preisliste S. 7
[A1 Breite T60-230] [A2 Höhenraster 18] [A3 Stil-Linie + PG Glatt2]
```

Der Kommentar in `KalkulationsPanel.tsx` erklärt, warum das so aussieht:

Damit ist jeder Betrag ohne Rückfrage prüfbar: Der Bearbeiter sieht, welcher Artikel gezogen wurde und über welche Achsenwerte — nicht nur, dass „irgendwas 348 €“ kostet. Das ist der Unterschied zwischen einem Werkzeug, dem der Berater vertraut, und einer Blackbox.

Schritt 8 — Abschluss

Der Berater tut	Funktion	Ergebnis
„Skizze via Smartphone scannen“	<code>buildMobileScanUrl() + QRCode</code>	Handy öffnet <code>/scan/CRAMER-2026-AB-0001</code>
Foto machen	<code>MobileScan.tsx → postScan()</code>	Bild optimiert und übertragen
„Berechneten Preis übernehmen“	<code>formatDezimal(5547.3)</code>	VK-Feld: 5.547,30
„PDF herunterladen“	<code>downloadPdf(draft)</code>	AV-PDF, 2 Seiten
„An AV senden“	—	Gesperrt , solange Auftrags-/Artikelnummer fehlt
„Speichern & abschließen“	<code>finalizeDraft()</code>	Entwurf erhält Abschlussdatum

Und was ist mit einem Atrium-Schrank?

Die ursprüngliche Frage nannte ein Atrium-Beispiel. Ehrlich beantwortet:

Bereich	Verhalten bei Atrium
Serien- und Produktauswahl	Funktioniert vollständig — 83 Artikel freigegeben
Korpus-Materialien	Funktioniert vollständig (mit Abdeckplatte, die es bei Refugium nicht gibt)
Maße	Einfache H/B/T-Eingabe mit Segment-Zähler

Bereich	Verhalten bei Atrium
Fronten inkl. Griffe	Funktioniert vollständig — und wird auch bepreist , weil die Front- und Griff-Zuordnungen serienunabhängig sind
Innenausstattung	Wird bepreist, soweit Zuordnungen bestehen
Korpus, Mittelseite, Außenset	Werden nicht bepreist. In <code>preisMapping.ts</code> fehlt für Atrium die Bauteil-Zuordnung.
Meldung	<i>„Für die Serie atrium sind noch keine Bauteil-Zuordnungen hinterlegt (config/preisMapping.ts). Die Kalkulation bleibt unvollständig.“</i>
Zusammenfassung, PDF, Speichern	Funktionieren vollständig

[EMPFEHLUNG] Für Atrium wären in `preisMapping.ts` drei Zeilen zu ergänzen: Korpus 10-10-05-0001 (Achsen Breite × Raster × PG × Tiefe × Variante), Mittelseite 10-10-10-0001 und Außenset 10-10-15-0001. Der KorpushöhenOffset für Atrium (52 mm) steht bereits in der Excel. Das ist ein kleiner, klar umrissener Arbeitsschritt — kein Umbau.

KAPITEL 8

Wenn der Kunde fragt ...

„Woher kommen die Artikel im Dropdown?“

Das hängt vom Dropdown ab, und diese Unterscheidung sollte man kennen. Die **Preise und Artikeldaten** kommen zu 100 % aus der Excel-Mappe — kein einziger Preis steht im Programmcode. Die **Auswahllisten für Materialien, Front-Typen, Stil-Linien, Griffe und Ausstattung** kommen heute noch aus Katalogdateien im Code. Die Technik, um auch diese Listen direkt aus der Excel zu speisen, ist fertig gebaut und getestet — sie ist nur noch nicht an die Auswahlfelder angeschlossen.

„Müssen neue Artikel programmiert werden?“

Für den Artikel selbst und seinen Preis: **nein**. Eine Zeile in „10 Artikel“, eine Zeile in „20 Preise“ — fertig. Damit ist er im System, in der Suche, in der Klassifikation und bepreisbar. Damit er auch in einem der genannten Auswahlkataloge auftaucht, braucht es heute noch eine kleine Code-Ergänzung. Bei Griffen ist die Verbindung schon automatisch: Die Kalkulation findet einen neuen Griff über die Nummer im Namen.

„Wie wird ein neuer Griff ergänzt?“

Eine Zeile im Blatt „10 Artikel“: Artikelnummer aus dem Nummernkreis 30-30-05- (die App schlägt über „Duplizieren“ automatisch die nächste freie vor), Bezeichnung „Griff Nr. 130“, Teileart BESCHLAG, Produktgruppe FRONTAUSSTATTUNG, Artikelgruppe GRIFF, Modus mit den Serienbuchstaben, Status **aktiv**. Dazu eine Zeile im Blatt „20 Preise“ mit dem Betrag. Das geht wahlweise in Excel oder direkt in der Artikelverwaltung der laufenden App.

„Wie wird ein Artikel gesperrt?“

Status von **aktiv** auf **gesperrt** setzen — ein Feld, kein Löschen. Der Artikel verschwindet sofort aus allen Auswahllisten. In bereits bestehenden Entwürfen verschwindet er nicht still: Dort steht die Position weiterhin, aber mit dem Vermerk „auf Anfrage — Artikel ist gesperrt“. Der Vorteil gegenüber dem Löschen: Historische Entwürfe und Aufträge bleiben nachvollziehbar.

„Wo werden Preise gepflegt?“

Ausschließlich im Blatt „20 Preise“ der Stammdatenmappe — oder gleichwertig im Reiter „Preisblatt“ der Artikelverwaltung. Jede Preiszeile trägt neben dem Betrag ihre Achsenwerte (Breite, Raster, Preisgruppe, Tiefe, Variante) und die Seite der gedruckten Preisliste als Herkunftsnachweis. Im Programmcode steht kein einziger Preis.

„Woher weiß das System, welcher Griff zu welchem Möbel passt?“

Über das Feld „Modus“ am Artikel. Dort stehen die Kürzel der Serien, für die er freigegeben ist — ein Großbuchstabe bedeutet Standard, ein Kleinbuchstabe Sonderanfertigung. Bei den Griffen steht überall **AVPROCSTU**, also alle neun Serien. Trüge ein Griff nur **R**, wäre er ausschließlich für Refugium sichtbar. Zusätzlich gibt es Regeln aus der Fachdokumentation, etwa dass bei den Stil-Linien Glossy und Less vier bestimmte Griffnummern konstruktiv nicht möglich sind.

„Kann man neue Möbelfamilien ergänzen?“

Eine neue **Serie** ist eine Zeile im Blatt „30 Programme“. Damit sie auf einer Kachel erscheint und automatisch bepreist wird, sind heute zwei kleine Code-Ergänzungen nötig: die Zuordnung zur Produktfamilie und die

Bauteilzuordnung fürs Preisblatt. Das ist überschaubarer Aufwand — zwischen fünf und zwanzig Zeilen —, aber es ist Entwicklerarbeit, keine Datenpflege.

„Was passiert, wenn sich Preise ändern?“

Betrag in der Excel ändern, Programm neu erzeugen — beim nächsten Start rechnet alles mit den neuen Preisen. Alternativ direkt in der Artikelverwaltung ändern; dann wirkt die Änderung sofort, auch in einer bereits geöffneten Kalkulation. Bereits **abgeschlossene** Entwürfe behalten ihren gespeicherten VK-Preis; das PDF ist ohnehin bereits erzeugt.

„Was ist im Code fest hinterlegt?“

Die Rechenwege und Regeln: die Raster-Formeln, die „nächstgrößeres Maß“-Regel, die Reihenfolge der Schritte, die Pflichtfeld-Prüfungen, die Ausschlussregeln (PTO/Griff, zweiläufige Schiebetür). Dazu die Auswahl-Kataloge für Farben, Front-Typen, Stil-Linien, Griffe und Ausstattung. **Nicht** im Code: Artikel, Preise, Preisgruppen-Definitionen, Serien, Zuschlagssätze, MwSt., Rastermaß, Höhen-Offsets, Berater, Filialen.

„Was kann der Kunde später selbst pflegen?“

Alles, was Artikel und Preis betrifft: neue Artikel anlegen, Bezeichnungen ändern, Preise ändern, Artikel sperren, Serien-Freigaben ändern, Reihenfolge im Dropdown ändern, Berater und Filialen verwalten, MwSt.- und Zuschlagssätze ändern. Dafür gibt es drei Wege: die Excel-Mappe, die Artikelverwaltung in der laufenden App, oder Export→Bearbeiten→Import. In der Artikelverwaltung ist ein Handbuch eingebaut, das genau die Frage beantwortet: „Was muss ich eintragen, damit der Konfigurator meinen neuen Artikel anbietet?“

„Wie verhindert der Planer falsche Konfigurationen?“

Auf sechs Ebenen: Erstens erscheinen unzulässige Optionen gar nicht erst. Zweitens schließen sich bestimmte Auswahlen technisch aus (PTO und Griff sind nicht gleichzeitig anklickbar). Drittens ist der „Weiter“-Knopf gesperrt, solange Pflichtfelder fehlen — und wer per Adresszeile springt, wird zurückgeschickt. Viertens meldet das System Unstimmigkeiten im Klartext, etwa wenn die Segmentbreiten nicht zur Gesamtbreite passen. Fünftens rät die Kalkulation nie: Fehlt ein Preis, steht dort „auf Anfrage“ mit Begründung, und der Preis gilt als nicht verbindlich. Sechstens verweigert die Datenpflege inkonsistente Zustände — eine Filiale mit zugeordneten Mitarbeitern lässt sich nicht löschen.

„Wo werden abgeschlossene Konfigurationen gespeichert?“

Aktuell: **im Browser des jeweiligen Geräts** (localStorage). Es gibt keine Datenbank und keinen Server. Konkret heißt das: Entwürfe von Berater A auf Laptop A sind auf Laptop B nicht sichtbar, und ein Leeren der Browserdaten löscht sie. Das ist eine bewusste Prototyp-Entscheidung und im Code an mehreren Stellen ausdrücklich als solche markiert. Der dauerhafte Ausgang ist heute das AV-PDF plus E-Mail.

„Wie könnte das System später an ein ERP angebunden werden?“

Der Weg ist bereits vorgezeichnet, und das ist die eigentliche Leistung der Artikelnummern-Logik. Jede Position der Kalkulation trägt bereits eine ERP-taugliche Artikelnummer, eine Menge, einen Einzelpreis und die Klassifikation. Ein Entwurf ist vollständig als JSON-Datensatz abgelegt — also in genau dem Format, das Schnittstellen erwarten. Praktisch wären drei Dinge zu tun: die Artikelnummern mit dem ERP abgleichen, ein Backend statt des Browser-Speichers einführen, und statt der PDF-E-Mail einen Aufruf der ERP-Schnittstelle einbauen. Die Datenstruktur selbst müsste nicht angefasst werden.

„Was müsste geändert werden, wenn Excel später durch eine Datenbank ersetzt wird?“

Erstaunlich wenig — und zwar mit Absicht. Der gesamte Rest des Programms greift auf Stammdaten ausschließlich über **eine** Datei zu (`src/lib/stammdaten.ts`) bzw. über den Arbeitsstand (`src/lib/stammdatenStore.ts`). Diese beiden Dateien sind die einzige Stelle, die wüsste, dass die Daten künftig aus einer Datenbank kommen. Kalkulation, Preis-Lookup, Auswahllisten und PDF würden unverändert weiterlaufen, weil sie die Datenherkunft gar nicht kennen. Der Kommentar im Zugangs-Speicher sagt es beispielhaft: *„Mit dem Backend zieht dieser Store an den Server; die Schnittstelle bleibt dieselbe.“* Praktisch wären zu tun: die Ladefunktionen auf Datenbankabfragen umstellen, die Änderungs-Folie durch echte Schreibvorgänge ersetzen, und den Excel-Import als Migrationswerkzeug behalten.

„Läuft das offline? Und braucht es Internet im Kundengespräch?“

Die Anwendung selbst läuft vollständig im Browser und braucht nach dem Laden keine Verbindung zum Server — alle Daten und Regeln sind lokal. Internet bzw. WLAN wird nur an zwei Stellen gebraucht: für den Smartphone-Scan der Handskizze (beide Geräte müssen sich erreichen) und für den E-Mail- Versand an die AV.

„Wer darf was?“

Zwei Rollen. **Berater** kommen auf Dashboard und Konfigurator. **Administratoren** kommen zusätzlich auf das Admin-Dashboard mit der Stammdaten- und Artikelverwaltung, der Benutzerverwaltung und dem Wartungsmodus. Der Wartungsmodus blendet für alle Berater ein Hinweis-Overlay ein; Login und Admin-Bereich bleiben erreichbar, damit man ihn wieder abschalten kann.

„Wie sicher ist das?“

Für einen Prototyp angemessen, für den Produktivbetrieb nicht ausreichend — und das steht so auch im Code. Passwörter werden zwar als Hash gespeichert und nie im Klartext, aber es gibt kein Backend, keine zentrale Benutzerverwaltung und keine Verschlüsselung der Entwürfe. Für die Produktion ist eine echte Anmeldung über Backend oder Single Sign-On vorgesehen. Positiv: Es war eine bewusste Entscheidung, **keine Passwörter in die Excel-Mappe** zu schreiben — die Mappe geht per Mail und OneDrive herum.

KAPITEL 9

Glossar

Begriff	Erklärung für Nicht-Programmierer
Frontend	Der Teil, den man sieht und bedient: Seiten, Knöpfe, Auswahllisten. Beim CRAMER PLANER läuft das komplett im Browser.
Backend	Der Teil, der im Hintergrund auf einem Server läuft. Der CRAMER PLANER hat heute keines — deshalb liegen alle Daten auf dem jeweiligen Gerät.
Datenbank	Ein zentraler Datenspeicher, auf den viele Nutzer gleichzeitig zugreifen. Hier ersetzt durch die Excel-Mappe (Produktdaten) und den Browser-Speicher (Entwürfe).
Datensatz	Eine Zeile in einer Tabelle. Ein Artikel = ein Datensatz mit 22 Feldern.
Feld	Eine Spalte innerhalb eines Datensatzes, z. B. „Modus“ oder „Status“.
Variable	Ein benannter Zwischenspeicher im Programm, z. B. „die gerade gewählte Serie“.
Function (Funktion)	Ein kleiner Arbeitsauftrag: bekommt Informationen, verarbeitet sie, liefert ein Ergebnis. „ <i>Hier ist die Höhe 230 cm — sag mir die Rasterstufe.</i> “ → „18.“
Parameter	Die Information, die man einer Funktion mitgibt (hier: 230 cm).
Rückgabewert	Das Ergebnis, das die Funktion liefert (hier: 18).
API / Schnittstelle	Eine vereinbarte Übergabestelle zwischen zwei Systemen. Im CRAMER PLANER gibt es zwei kleine: <code>/api/scan</code> für den Smartphone-Scan und <code>/api/net-host</code> für die QR-Adresse.
Filter	Eine Auswahlbedingung, die aus einer großen Liste eine kleinere macht: „ <i>nur Artikel mit R im Modus und Status aktiv.</i> “
Boolean	Ein Ja/Nein-Wert. Beispiel: <code>oberflaeche: true, pto: false</code> .
ID	Eine eindeutige Kennung. Im CRAMER PLANER: die Artikelnummer, die Entwurfsnummer, die Personalnummer, die Filialnummer.
Hardcoded (fest verdrahtet)	Ein Wert steht direkt im Programmcode und lässt sich nur durch Programmieren ändern. Beispiel hier: die Farbliste.
Datengetrieben	Das Verhalten ergibt sich aus Daten, nicht aus Programmcode. Beispiel hier: alle Preise und Artikel.
Regelbasiert	Das System entscheidet anhand hinterlegter Regeln, was möglich ist — statt alles zuzulassen und hinterher zu prüfen.
JSON	Ein einfaches, textbasiertes Datenformat, das fast jedes System lesen kann. Alle Entwürfe liegen als JSON vor — das macht eine spätere ERP-Anbindung deutlich leichter.
localStorage	Der Speicher des Browsers auf dem jeweiligen Gerät. Bleibt beim Schließen erhalten, ist aber gerätegebunden.
Build / „erzeugen“	Der Vorgang, aus Quelldateien die lauffähige Anwendung zu machen. Hier wird dabei auch die Excel eingelesen.
Deployment	Das Ausrollen einer neuen Programmversion. Für Excel-Änderungen nötig, für Änderungen in der Artikelverwaltung nicht.
Prototyp	Eine funktionsfähige Vorstufe, die den fachlichen Ablauf beweist, aber technische Vereinfachungen enthält.

— Cramer-spezifisch —

Begriff	Erklärung für Nicht-Programmierer
Modus	Das Feld am Artikel, das die Serien-Freigabe trägt. GROSS = Standard, klein = Sonderanfertigung.
Achse (A1–A5)	Eine Dimension, entlang derer sich der Preis verändert: Breite, Raster, Preisgruppe, Tiefe, Variante, Bedingung, Stil-Linie+PG.
Preiszelle / Preiszeile	Ein konkreter Preis für eine bestimmte Kombination von Achsenwerten.
Raster	Cramers Höheneinheit: 1 Raster = 12,8 cm. Die Preisliste ist darauf geschlüsselt.
Bracket	Eine Maßstufe der Preisliste („bis 60 cm“). Ein Individualmaß wird immer auf das nächstgrößere Bracket gehoben.
Teileart	Die Sicht von Einkauf und Fertigung: Struktur, Front, Beschlag, Ausstattung, Technik, Fertigmöbel, Kalkulation.
Produktgruppe (Excel)	Ein Schritt im Konfigurator (KORPUS, FRONT, ...).
Artikelgruppe	Ein Dropdown innerhalb eines Schritts (Griff, Drehtür, Container, ...).
Preislogik	Wie der Preis eines Artikels zustande kommt: Festpreis, Matrix, pro laufendem Meter, pro m ² , Prozent vom Möbelpreis, auf Anfrage ... (12 Varianten).
AV	Arbeitsvorbereitung — der Empfänger des PDF.
Overlay / Änderungs-Folie	Die im Admin-Bereich gemachten Änderungen, gespeichert als Abweichung vom Grundstand.
Entwurf (Draft)	Der zentrale Datensatz einer Konfiguration. Alles, was der Berater eingibt, landet hier.
Sondermaß	Ein Maß, das die Preisliste nicht kennt. Führt zu „auf Anfrage“ und AV-Prüfung — nie zu einem Schätzpreis.

KAPITEL 10

Die kompakte 2-Minuten-Erklärung

Zum Vorlesen oder freien Vortragen im Kundengespräch.

Der CRAMER PLANER ist eine Web-Anwendung, die auf Laptop oder iPad direkt im Kundengespräch läuft. Sie digitalisiert die Schnittstelle zwischen Beratung und Arbeitsvorbereitung — mit dem Ziel, dass keine Information mehr auf dem Weg verloren geht.

Der wichtigste Gedanke dahinter: **Wir konfigurieren keine fertigen Möbel, sondern setzen ein Möbel aus einzelnen Artikeln zusammen — wie bei einem Fahrzeugkonfigurator.** Aus einem Kleiderschrank mit vier Segmenten werden im System siebzehn einzelne Positionen: vier Korpusse, drei Mittelseiten, ein Außenset, fünf Fronten, ein Griff und die Innenausstattung. Jede Position hat ihre eigene Artikelnummer und ihren eigenen Preis aus der Preisliste.

Technisch trennen wir dabei streng zwischen **Programm** und **Daten**. Im Programm stehen die Regeln und Rechenwege: Wie rechnet man Zentimeter in Rasterstufen um? Welche Auswahl schließt welche aus? Wann ist ein Schritt vollständig? Die Produktwelt selbst — 190 Artikel, 1.509 Preise, neun Serien, alle Mitarbeiter und Filialen — steht in einer einzigen Excel-Mappe. Kein Preis, kein Artikel und keine Serie steht im Programmcode.

Diese Trennung hat einen unmittelbaren praktischen Nutzen: **Ein neuer Griff, ein geänderter Preis oder eine neue Serien-Freigabe ist eine Zeile in Excel — nicht ein Auftrag an einen Programmierer.** Und weil jeder Artikel eine sprechende Nummer trägt — die vier Blöcke sagen Teileart, Konfigurator-Schritt, Dropdown und laufende Nummer —, ist die Datenbasis von Anfang an ERP-tauglich aufgebaut.

Damit die Beratung sauber bleibt, arbeitet der Planer regelbasiert. Unzulässige Optionen erscheinen gar nicht erst: Ein Refugium-Kleiderschrank zeigt keine Abdeckplatten-Auswahl, weil in den Stammdaten kein einziger Abdeckplatten-Artikel für Refugium freigegeben ist. Push-to-Open und Griff lassen sich technisch nicht gleichzeitig anklicken. Und weiter geht es erst, wenn alle Pflichtangaben stehen.

Beim Preis gilt ein Grundsatz, der uns wichtig ist: **Das System rät nie.** Wenn die Preisliste für ein Maß keine Zeile hat — etwa für eine 100 cm breite Drehtür —, dann erscheint nicht irgendein hochgerechneter Betrag, sondern „auf Anfrage“ mit der Begründung im Klartext. Und jeder Betrag, der angezeigt wird, ist belegt: Neben jeder Position stehen Artikelnummer, die verwendeten Maßstufen und sogar die Seite der gedruckten Preisliste. Kein Berater muss einer Blackbox vertrauen. Der verbindliche Verkaufspreis bleibt am Ende die Entscheidung des Beraters — die Kalkulation liefert einen belegten Vorschlag.

Am Ende steht das AV-PDF: alle Angaben auf Seite eins, die per Smartphone abfotografierte Handskizze auf Seite zwei.

Zum Stand: Der Prototyp ist funktional vollständig — von der Anmeldung bis zum PDF. Zwei Dinge sind für den Produktivbetrieb noch zu tun. Erstens braucht es einen Server, damit Entwürfe nicht nur lokal auf einem Gerät liegen, sondern zentral und für alle Filialen. Und zweitens sind die Bauteil-Zuordnungen fürs Preisblatt bisher für Refugium vollständig hinterlegt — die übrigen Serien folgen demselben Muster und sind ein überschaubarer, klar umrissener Arbeitsschritt.

Anhang: Die wichtigsten Prüf- und Pflegebefehle

```

npm run dev           # Anwendung starten (erzeugt vorher die Stammdaten neu)
npm run data:build    # Stammdaten-Modul aus Excel + Markdown neu erzeugen
npm run data:check    # Nur Kreuzprüfung Excel ↔ Markdown ↔ Verweise, nichts schreiben
npm run data:test     # Selbsttest der Modus- und Lookup-Logik gegen die echten Daten
npm run kalk:test     # Kalkulation gegen die Sollwerte des Vorgänger-Tools
npm run typecheck     # Prüft die Typ-Konsistenz des gesamten Codes

```

Anhang: Zahlen zum aktuellen Datenbestand

Kennzahl	Wert
Artikel	190 (alle Status aktiv)
Preiszeilen	1.509 (1.503 fest · 4 auf Anfrage · 2 mit Hinweis)
Serien	9
Produktgruppen (= Konfigurator-Schritte)	9
Artikelgruppen (= Dropdowns)	38
Teilearten	7
Preislogiken	12
Achsen	7
Mitarbeiter / Filialen	4 / 6
Preisliste gültig ab	06.2026
Rastermaß	128 mm (12,8 cm)
MwSt.	19 % (alle Preise inkl.)
Montage-Zuschlag / Lieferung regional	10 % / 3 %